RWANDA POLYTECHNIC

Build Machine-Learning-Ready Datasets - Practical Task (Assignment 1)

Question 2 - Own Rwandan Scenario

Predicting Patient Waiting Times at Rwandan Health Facilities

Documentation Report: code, outputs and charts for Stages 0 to 6 (PC 1.1 - 1.4)

Item	Detail
Group	Group 4
Member 1	Jean Luc Tuyizere - 26RP01265
Member 2	Jean De Dieu Mugisha - 26RP01151
Target	`waiting_time_min` (regression). No prediction model is trained.
Notebook	`q2_own_scenario/notebooks/q2_waiting_time_data_preparation.ipynb`
Date submitted	_____
Lecturer	_____

Contents

Contents....2
About this report....4
Stage 0 - Scenario....5
0.1 Scenario brief (illustrative case study written by the group)....5
0.2 Scenario Proposal Form....5
0.3 Business questions the prepared data must answer....6
Stage 1 - Configure the environment (PC 1.1)....7
Stage 2 - Requirements and acquisition (PC 1.2)....8
2.1 Data requirements statement....8
2.2 Source comparison....8
2.3 Keep the original file unchanged and print its MD5....8
2.4 Load the data into pandas....9
2.5 Requirement check....10
2.6 Data Dictionary....11
2.7 Source Log....12
Stage 3 - EDA and data audit (PC 1.3)....14
3.1 Missing values, including disguised ones....16
3.2 Duplicates....17
3.3 Inconsistent spellings and formats....18
3.4 Impossible and illogical values....19
3.5 Outliers: box plots and the IQR rule....20
3.6 Target study....21
3.7 Relationships (charts 4 to 7)....22
3.8 Leakage check....26
3.9 Data Quality Report....26
Stage 4 - Clean and pre-process (PC 1.3)....29
4.1 Tidy the text labels....29
4.2 Remove duplicates....30
4.3 Correct types, formats and units....30
4.4 Rebuild facility_type from the facility code....31
4.5 Set impossible values to missing....31
4.6 Decide what to do with outliers....32
4.7 Drop leakage and identifier columns....32
4.8 Remove records with no target....33
4.9 Save the clean file and show the Cleaning Log....34
4.10 Split 70 / 15 / 15 before imputing or scaling....35

4.11 Impute with values learned from the training set only 35
4.12 Encode the categories 37
4.13 Is the target imbalanced? 38
Stage 5 - Features (PC 1.4) 40
5.1 Create new features 40
5.2 Skewness and transformations (training set) 42
5.3 Scale the continuous columns (scaler fitted on the training set only) 43
5.4 Relationship with the target (training set): correlation and mutual information 44
5.5 Feature selection (decided on the training set only) 46
5.6 Feature Justification table 48
5.7 Apply identical steps to train, validation and test 51
Stage 6 - Finalise, package and present (PC 1.1 - 1.4) 52
6.1 Save the final files and the fitted pipeline 52
6.2 prepare_new_data(): from raw records to model-ready features 52
6.3 Definition of done (computed in code) 54
6.4 MD5 fingerprints of the final files 55
6.5 Data Card 55
Summary of deliverables 58
Final deliverables checklist 59

About this report

This report documents Question 2 from start to finish. It is built directly from one complete run of the notebook, so every code block, output table and chart shown here is the real result of that run and matches the notebook cell by cell.

How to read it. Each step appears in the same order as in the notebook:

- **Code** (grey block): the code of the cell.
- **Output** (blue block, table or chart): what the cell produced when it was run.
- **Explanation** (normal text): what the step does, what was found and what was decided.

How the stages map to the performance criteria.

Stage	Content	Performance criterion
Stage 0	Scenario brief, Scenario Proposal Form, business questions	Q2 scenario
Stage 1	Configure the environment	PC 1.1
Stage 2	Requirements and data acquisition	PC 1.2
Stage 3	Exploratory data analysis and data audit	PC 1.3
Stage 4	Cleaning and pre-processing	PC 1.3
Stage 5	Feature engineering and selection	PC 1.4
Stage 6	Finalise, package and present	PC 1.1 - 1.4

Items still open. The link, date accessed and licence of the data source, a second source for the comparison table, and the lecturer's signature are left blank and are marked in the text.

Stage 0 - Scenario

0.1 Scenario brief (illustrative case study written by the group)

This scenario is a teaching case written by the group for this assignment. No real request was made by any institution.

Who is asking. The quality-improvement manager of a health planning team at the Ministry of Health (MoH) in Kigali. The manager wants to start with a pilot in **Nyabihu District, Western Province**, and later extend the work to other districts.

The field. Public health service delivery: outpatient visits at health centres, district hospitals and referral hospitals.

The problem. Patients complain that they wait a long time before they are seen. Long waits push patients to leave without care, crowd the waiting areas and add pressure on staff. Managers currently learn about long waits only after they happen.

The decision a prediction would support. If the waiting time of a visit could be estimated at the moment the patient arrives, a facility manager could (a) move staff to the departments and hours where long waits are expected, (b) advise scheduled patients to come at quieter hours, and (c) tell patients honestly how long they are likely to wait.

What data exists. One file of 1,200 visit records with 20 columns. It covers 22 districts in all five provinces from January 2024 to December 2025. Each record describes the facility, the visit date and arrival hour, the patient (age, sex, insurance), the department, the type of appointment, the distance and travel time, the number of earlier visits, the waiting time, the consultation time and whether the patient attended.

What the manager asks us to deliver. Not a model. The manager asks for a clean, documented, model-ready dataset (train, validation and test files), a reusable preparation function, and clear documents (data dictionary, source log, data quality report, cleaning log, feature justification, data card) so that an analyst can later train and test a model without redoing the preparation.

Target variable. `waiting_time_min` - the minutes a patient waits between arrival and the start of the consultation. It is a number, so this is a **regression** problem. Predicting it matters because it is the measure patients experience directly and the one managers can act on by changing staffing and scheduling.

True origin of the data. The group states that the file comes from NISR (National Institute of Statistics of Rwanda). This has not been verified: no link, download record or licence text is on file. The link, date accessed and licence are still to be added (see the Source Log in section 2.7). Until they are, nobody should present the records as verified Rwandan patient visits.

0.2 Scenario Proposal Form

Field	Entry
Group	**Group 4**
Members	Jean Luc Tuyizere (26RP01265); Jean De Dieu Mugisha (26RP01151)
Title	Predicting patient waiting times at Rwandan health facilities
Location	Pilot: Nyabihu District, Western Province. Data covers 22 districts in 5 provinces.
Stakeholder	Quality-improvement manager, health planning team, Ministry of Health (illustrative)
Story and decision	Patients at health centres and hospitals often wait a long time before they are seen. The manager wants to know, at the moment a patient arrives, how long that patient is likely to wait. With that estimate a facility can move staff to busy departments and hours. It can also advise scheduled patients to come at quieter times. The manager first needs a clean, well-documented dataset, because the visit records have gaps, repeated rows and untidy text. Our task is to prepare that dataset so that a model can be trained later. We do not train the model.
Who benefits	Patients (shorter and more predictable waits), facility managers

	(staff planning), district and national planners (comparison between facilities)
Target	`waiting_time_min` (minutes waited before consultation)
Problem type	Regression
Data source and link	NISR (National Institute of Statistics of Rwanda), as stated by the group; file `rwanda_health_facility_visits.csv`. **Link, date accessed and licence: TO BE COMPLETED BY THE GROUP**
Expected rows and columns	Raw: 1,200 rows x 20 columns. Usable rows with a waiting time after cleaning: about 1,027
Known risks - privacy	No names, phone numbers or national IDs. `visit_id` and `facility_code` are record and facility codes; both are removed from the final files. Age, sex, district and visit date together could narrow down a person if the records are real.
Known risks - licence	Licence not on file. The NISR terms of use for this dataset must be checked and recorded.
Known risks - missing data	14 of 20 columns have gaps. 168 records have no waiting time and cannot be used.
Lecturer approval signature	_____ Date: _____

0.3 Business questions the prepared data must answer

No.	Business question
BQ1	Which facility levels and service departments have the longest waiting times?
BQ2	How does the time of the visit (arrival hour, day of week, month) relate to waiting time?
BQ3	Does the patient's location and access (province, district, distance, travel time) relate to waiting time?
BQ4	Do patient characteristics (age group, sex, insurance, earlier visits) relate to waiting time?
BQ5	Does the way the patient enters care (walk-in, scheduled, follow-up, referral) change the waiting time?

These questions are used in Stage 5 to justify every feature.

Stage 1 - Configure the environment (PC 1.1)

Code - cell 1

```
import hashlib
import platform
import re
import shutil
import unicodedata
from pathlib import Path

import joblib
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import sklearn
from IPython.display import Markdown, display
from sklearn.feature_selection import SelectKBest, f_regression, mutual_info_regression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

SEED = 42
np.random.seed(SEED)
pd.set_option("display.max_columns", 60)
pd.set_option("display.width", 200)
sns.set_theme(style="whitegrid")

# Folders. The notebook lives in q2_own_scenario/notebooks, so the project root is one level up.
ROOT = Path.cwd() if (Path.cwd() / "data").exists() else Path.cwd().parent
RAW_DIR, INTERIM_DIR, PROCESSED_DIR, DOCS_DIR = (ROOT / "data" / "raw", ROOT / "data" / "interim",
                                                 ROOT / "data" / "processed", ROOT / "docs")
for folder in (RAW_DIR, INTERIM_DIR, PROCESSED_DIR, DOCS_DIR):
    folder.mkdir(parents=True, exist_ok=True)

RAW_NAME = "rwanda_health_facility_visits.csv"
RAW_FILE = RAW_DIR / RAW_NAME
TARGET = "waiting_time_min"


def md5(path):
    """Return the MD5 fingerprint of a file. Two files with the same MD5 have the same bytes."""
    return hashlib.md5(Path(path).read_bytes()).hexdigest()


print("Python      :", platform.python_version())
for name, module in (("pandas", pd), ("numpy", np), ("matplotlib", matplotlib), ("seaborn", sns),
                     ("scikit-learn", sklearn), ("joblib", joblib)):
    print(f"{name:<12}: {module.__version__}")
print("Random seed :", SEED)
print("Project root:", ROOT.name)
print("Folders     :", [str(f.relative_to(ROOT).as_posix()) for f in (RAW_DIR, INTERIM_DIR, PROCESSED_DIR, DOCS_DIR)])
```

Output

```
Python      : 3.13.0
pandas      : 2.2.3
numpy       : 2.2.5
matplotlib  : 3.10.1
seaborn     : 0.13.2
scikit-learn: 1.7.0
joblib      : 1.5.1
Random seed : 42
Project root: q2_own_scenario
Folders     : ['data/raw', 'data/interim', 'data/processed', 'docs']
```

What this does. Loads the allowed libraries (pandas, numpy, matplotlib, seaborn, scikit-learn, joblib) plus small tools that come with Python and Jupyter, and prints the version of each one. It fixes the random seed at 42, finds the project folder and makes sure `data/raw`, `data/interim`, `data/processed` and `docs` exist. The `md5()` helper gives a fingerprint of a file so we can prove that a file did not change.

What we decided. The environment is plain Python with Jupyter (no conda). The same versions are listed in `requirements.txt` so that another person can install the same environment with `pip install -r requirements.txt`. One seed (42) is used for every random step.

Stage 2 - Requirements and acquisition (PC 1.2)

2.1 Data requirements statement

Requirement	What we need
Target	The waiting time of each visit in minutes (`waiting_time_min`).
Features that could influence it	Facility level and department (BQ1); visit date, day and arrival hour (BQ2); province, district, distance and travel time (BQ3); age, sex, insurance and earlier visits (BQ4); appointment type and referral (BQ5).
Time period	At least 12 months, so that every month of the year is covered.
Area	Rwandan districts, including the pilot district (Nyabihu).
Number of records	At least 1,000 records and 8 columns (task minimum), with numerical and categorical columns.
Quality	Real quality problems must be present (the task does not accept perfectly clean data).
Privacy	No names, phone numbers or national IDs.

2.2 Source comparison

Criterion	Source A - `rwanda_health_facility_visits.csv` (chosen)	Source B
Relevance	High: one row per visit, with a waiting time and the facility, time, location and patient columns the business questions need.	None - the group did not consider a second source
Size	1,200 rows x 20 columns.	Not assessed
Licence	**Not on file - TO BE COMPLETED BY THE GROUP**	Not assessed
Quality	Has missing values, duplicate rows, untidy text and outliers (measured in Stage 3).	Not assessed
Reason for the choice	It is the only source the group considered.	-

Gap to note: the task asks for at least two sources to be compared. The group considered only one, so this table cannot show a real comparison. We did not invent a second source. To meet the requirement, the group should look at one real alternative and fill in the right-hand column.

2.3 Keep the original file unchanged and print its MD5

Code - cell 2

```
ORIGINAL_FILE = ROOT.parent / RAW_NAME          # where the file was first placed
EXPECTED_RAW_MD5 = "fe54529b0294f43151a9f5b8815d01a7"   # fingerprint recorded when the file was received

if not RAW_FILE.exists():
    shutil.copy2(ORIGINAL_FILE, RAW_FILE)       # copy only; the original is never opened for writing

RAW_MD5_START = md5(RAW_FILE)
print("Raw file :", RAW_FILE.relative_to(ROOT))
print("Size     :", RAW_FILE.stat().st_size, "bytes")
print("MD5      :", RAW_MD5_START)
if ORIGINAL_FILE.exists():
    print("Same as the original file:", md5(ORIGINAL_FILE) == RAW_MD5_START)
assert RAW_MD5_START == EXPECTED_RAW_MD5, "The raw file is not the file we received."
```

Output

```
Raw file : data\raw\rwanda_health_facility_visits.csv
Size     : 186300 bytes
MD5      : fe54529b0294f43151a9f5b8815d01a7
Same as the original file: True
```

What this does. Copies the original file into `data/raw/` if it is not already there, then prints its MD5 fingerprint and checks it against the fingerprint recorded when we received the file.

What we found. The MD5 is `fe54529b0294f43151a9f5b8815d01a7` and it matches the original file.

What we decided. The raw file is never edited. Every change from here on is made in code on a copy held in memory. At the end of the notebook we check the MD5 again.

2.4 Load the data into pandas

Code - cell 3

```
# keep_default_na=False stops pandas from silently turning the text "None" or "NA" into a missing value.
# Only a truly empty cell is read as missing. This lets us see disguised missing values in Stage 3.
raw = pd.read_csv(RAW_FILE, encoding="utf-8", keep_default_na=False, na_values=[""])

print("Rows:", raw.shape[0], "| Columns:", raw.shape[1])
print()
print(raw.dtypes)
```

Output

```
Rows: 1200 | Columns: 20

visit_id                     object
province                     object
district                     object
facility_code                object
facility_type                object
visit_date                   object
patient_age_years           float64
sex                          object
insurance_type               object
service_department           object
appointment_type             object
arrival_hour                  int64
day_of_week                  object
distance_to_facility_km     float64
estimated_travel_time_min   float64
referral                     object
previous_visits_12m         float64
waiting_time_min            float64
consultation_duration_min   float64
attended                     object
dtype: object
```

What this does. Reads the raw CSV into a DataFrame called `raw` and prints the number of rows, the number of columns and the data type of each column.

What we found. 1,200 rows and 20 columns. Seven columns are numbers (`float64` or `int64`) and thirteen are text (`object`). `visit_date` is read as text and will need converting to a date.

What we decided. We load with `keep_default_na=False` because, by default, pandas reads the word "None" as a missing value. `insurance_type` contains the word "None", and we want to judge its meaning ourselves instead of letting pandas hide it.

Code - cell 4

```
display(raw.head())
display(raw.tail())
```

Output

```
       visit_id  province district facility_code      facility_type  visit_date  patient_age_years     sex     insurance_type  \
0  RW-HLTH-00591  Northern  Musanze     DH-MUS-03  District Hospital  2025-02-11               54.0  Female               RSSB
1  RW-HLTH-01167   Western  Nyabihu     HC-NYA-02      Health Centre  2025-01-17               29.0  Female  Mutuelle de Santé
2  RW-HLTH-01173   Western  Karongi     HC-KAR-01      Health Centre  2024-02-13               58.0    male  Mutuelle de Santé
3  RW-HLTH-00551   Western  Nyabihu     HC-NYA-02      Health Centre  2025-09-29               52.0    Male            Private
4  RW-HLTH-00067  Northern  Gicumbi     HC-GIC-02      Health Centre  2025-06-08               41.0    Male  Mutuelle de Santé

  service_department appointment_type  arrival_hour day_of_week
0          NCD Clinic          Walk-in            13     Tuesday
1           Outpatient        Follow-up            11      Friday
2           Emergency        Follow-up            11     Tuesday
3        Child Health        Follow-up             7      Monday
4        Child Health        Follow-up            10      Sunday

   distance_to_facility_km  estimated_travel_time_min referral  previous_visits_12m  waiting_time_min  consultation_duration_min attended
0                     11.7                       37.0      Yes                  3.0              47.0                       14.0      Yes
1                      2.8                       12.0       No                  1.0              20.0                       19.0      Yes
2                      4.8                       29.0       No                  0.0              15.0                        7.0      Yes
3                     19.4                       73.0       No                  1.0              23.0                       14.0      Yes
4                      0.3                        5.0       No                  2.0              12.0                        7.0      Yes

          visit_id  province district facility_code      facility_type  visit_date  patient_age_years     sex     insurance_type  \
1195  RW-HLTH-00307  Northern  Rulindo     HC-RUL-03      Health Centre  2024-09-26               42.0  Female  Mutuelle de Santé
1196  RW-HLTH-00343   Western  Karongi     DH-KAR-02  District Hospital  2025-07-31               58.0  Female  Mutuelle de Santé
1197  RW-HLTH-00267  Southern   Nyanza     HC-NYA-01      Health Centre  2025-12-19               32.0  Female  Mutuelle de Santé
1198  RW-HLTH-00163   Eastern  Kayonza     HC-KAY-02      Health Centre  2025-08-13                7.0    Male               None
1199  RW-HLTH-00835  Southern     Huye     RH-HUY-03  Referral Hospital  2024-01-20               24.0    Male  Mutuelle de Santé

     service_department appointment_type  arrival_hour day_of_week
1195          NCD Clinic        Follow-up             7    Thursday
1196                 NaN        Scheduled             7    Thursday
1197          Outpatient        Scheduled            12      Friday
1198          Outpatient        Scheduled            11   Wednesday
1199     Maternal Health          Walk-in             8    Saturday
```

```
      distance_to_facility_km  estimated_travel_time_min referral  previous_visits_12m  waiting_time_min  consultation_duration_min
attended
1195                      8.6                       35.0       No                  2.0              15.0                       19.0
yes
1196                      7.0                       55.0      NaN                  2.0              40.0                       21.0
Yes
1197                      6.4                       18.0       No                  0.0               4.0                       20.0
Yes
1198                      9.9                        NaN      Yes                  0.0              52.0                        4.0
Yes
1199                      2.6                       18.0       No                  2.0              71.0                       21.0
Yes
```

What this does. Shows the first five and last five records.

What we found. The file is one row per visit. Even in these few rows we can see untidy labels (for example `male` and `Male`, `yes` and `Yes`) and empty cells (`NaN`), including empty waiting times.

2.5 Requirement check

Code - cell 5

```
dates_for_check = pd.to_datetime(raw["visit_date"], errors="coerce")       # a temporary view, raw is not changed
months_covered = dates_for_check.dt.to_period("M").nunique()
n_numeric = raw.select_dtypes("number").shape[1]
n_text = raw.select_dtypes("object").shape[1]
n_districts = raw["district"].str.strip().nunique()
person_id_words = ["name", "phone", "national_id", "nid", "email", "address"]
person_id_cols = [c for c in raw.columns if any(w in c.lower() for w in person_id_words)]

requirement_check = pd.DataFrame([
    ["At least 1,000 rows", "all", f"{len(raw)} rows", len(raw) >= 1000],
    ["At least 8 columns", "all", f"{raw.shape[1]} columns", raw.shape[1] >= 8],
    ["Numerical and categorical columns", "all", f"{n_numeric} numerical, {n_text} text", n_numeric > 0 and n_text >
0],
    ["Clearly defined target", TARGET, f"{raw[TARGET].notna().sum()} records have a value", TARGET in raw.columns],
    ["Facility and department features (BQ1)", "facility_type, service_department", "present", True],
    ["Time features (BQ2)", "visit_date, day_of_week, arrival_hour", "present", True],
    ["Location and access features (BQ3)", "province, district, distance_to_facility_km, estimated_travel_time_min",
"present", True],
    ["Patient features (BQ4)", "patient_age_years, sex, insurance_type, previous_visits_12m", "present", True],
    ["Entry-route features (BQ5)", "appointment_type, referral", "present", True],
    ["Time period of at least 12 months", "visit_date",
     f"{dates_for_check.min().date()} to {dates_for_check.max().date()} ({months_covered} months)", months_covered >=
12],
    ["Area: several districts including Nyabihu", "district", f"{n_districts} districts",
     "Nyabihu" in set(raw["district"].str.strip())],
    ["Real quality problems present", "all",
     f"{int(raw.isna().sum().sum())} empty cells, {int(raw.duplicated().sum())} duplicate rows", raw.isna().sum().sum()
> 0],
    ["No personal identifiers", "all", f"columns that look like personal identifiers: {person_id_cols}",
len(person_id_cols) == 0],
], columns=["Requirement", "Column(s) that meet it", "Evidence", "Met"])
requirement_check["Result"] = requirement_check.pop("Met").map({True: "PASS", False: "FAIL"})
display(requirement_check)
```

Output

	Requirement	Column(s) that meet it	Evidence	Result
0	At least 1,000 rows	all	1200 rows	PASS
1	At least 8 columns	all	20 columns	PASS
2	Numerical and categorical columns	all	7 numerical, 13 text	PASS
3	Clearly defined target	waiting_time_min	1032 records have a value	PASS
4	Facility and department features (BQ1)	facility_type, service_department	present	PASS
5	Time features (BQ2)	visit_date, day_of_week, arrival_hour	present	PASS
6	Location and access features (BQ3)	province, district, distance_to_facility_km, e...	present	PASS
7	Patient features (BQ4)	patient_age_years, sex, insurance_type, previo...	present	PASS
8	Entry-route features (BQ5)	appointment_type, referral	present	PASS
9	Time period of at least 12 months	visit_date	2024-01-02 to 2025-12-30 (24 months)	PASS
10	Area: several districts including Nyabihu	district	22 districts	PASS
11	Real quality problems present	all	570 empty cells, 5 duplicate rows	PASS
12	No personal identifiers	all	columns that look like personal identifiers: []	PASS

What this does. Tests each requirement from section 2.1 against the loaded data and builds the requirement check table. The date conversion here is a temporary view; `raw` itself is not changed.

What we found. Every requirement passes. The file has 1,200 rows and 20 columns, 24 months of visits (2 January 2024 to 30 December 2025) and 22 districts including Nyabihu. 1,032 records have a waiting time. No column holds a name, phone number or national ID.

What we decided. The dataset meets the minimum requirements, so we continue. One point to watch: only 1,032 records have a target value, which is just above the 1,000-row minimum.

2.6 Data Dictionary

No documentation came with the file. **Every meaning below is our assumption from the column name and the values**, and is marked as such.

Code - cell 6

```
data_dictionary = pd.DataFrame([
    ["visit_id", "Code of the visit record (not a person)", "text", "-", "Format RW-HLTH-00001. Should be unique.",
"Assumed"],
    ["province", "Province of the facility", "text", "-", "Eastern, Kigali City, Northern, Southern, Western",
"Assumed"],
    ["district", "District of the facility", "text", "-", "22 districts", "Assumed"],
    ["facility_code", "Code of the health facility", "text", "-", "Prefix HC, DH or RH + district letters + number. The
same code can appear in different districts.", "Assumed"],
    ["facility_type", "Level of the facility", "text (ordered)", "-", "Health Centre < District Hospital < Referral
Hospital", "Assumed"],
    ["visit_date", "Date of the visit", "date", "YYYY-MM-DD", "2024-01-02 to 2025-12-30", "Assumed"],
    ["patient_age_years", "Age of the patient", "number", "years", "0 to 110 accepted (0 = infant under one year)",
"Assumed"],
    ["sex", "Sex of the patient", "text", "-", "Female, Male", "Assumed"],
    ["insurance_type", "Health insurance of the patient", "text", "-", "Mutuelle de Sante, RSSB, Private, None. We read
'None' as 'no insurance'.", "Assumed"],
    ["service_department", "Department that received the patient", "text", "-", "7 departments", "Assumed"],
    ["appointment_type", "How the visit was arranged", "text", "-", "Walk-in, Scheduled, Follow-up, Referral",
"Assumed"],
    ["arrival_hour", "Hour of the day the patient arrived", "whole number", "hour (24h clock)", "0 to 23 accepted;
observed 7 to 16", "Assumed"],
    ["day_of_week", "Day of the week of the visit", "text", "-", "Monday to Sunday", "Assumed"],
    ["distance_to_facility_km", "Distance from the patient's home to the facility", "number", "km", "Above 0 and up to
150 accepted", "Assumed"],
    ["estimated_travel_time_min", "Estimated time to travel to the facility", "number", "minutes", "Above 0 and up to
480 accepted", "Assumed"],
    ["referral", "Whether the patient was referred from another facility", "text", "-", "Yes, No", "Assumed"],
    ["previous_visits_12m", "Number of visits by the patient in the previous 12 months", "whole number", "visits", "0
to 100 accepted", "Assumed"],
    ["waiting_time_min", "TARGET. Minutes waited between arrival and consultation", "number", "minutes", "Above 0 and
up to 720 accepted. Empty when the patient did not attend.", "Assumed"],
    ["consultation_duration_min", "Length of the consultation", "number", "minutes", "Known only after the wait ends
(leakage, see 3.7)", "Assumed"],
    ["attended", "Whether the patient attended the visit", "text", "-", "Yes, No", "Assumed"],
], columns=["Column", "Meaning", "Data type", "Unit", "Allowed values / notes", "Basis of meaning"])
data_dictionary.insert(2, "Type in raw file", data_dictionary["Column"].map(raw.dtypes.astype(str)))
assert list(data_dictionary["Column"]) == list(raw.columns), "The dictionary must list every column once."
data_dictionary.to_csv(DOCS_DIR / "data_dictionary.csv", index=False)
display(data_dictionary)
```

Output

	Column	Meaning	Type in raw file	Data type	Unit	Allowed values / notes	Basis of meaning
0	visit_id	Code of the visit record (not a person)	object	text	-	Format RW-HLTH-00001. Should be unique.	Assumed
1	province	Province of the facility	object	text	-	Eastern, Kigali City, Northern, Southern, Western	Assumed
2	district	District of the facility	object	text	-	22 districts	Assumed
3	facility_code	Code of the health facility	object	text	-	Prefix HC, DH or RH + district letters + numbe...	Assumed
4	facility_type	Level of the facility	object	text (ordered)	-	Health Centre < District Hospital < Referral H...	Assumed

5	visit_date	Date of the visit	object	date	YYYY-MM-DD	2024-01-02 to 2025-12-30	Assumed
6	patient_age_years	Age of the patient	float64	number	years	0 to 110 accepted (0 = infant under one year)	Assumed
7	sex	Sex of the patient	object	text	-	Female, Male	Assumed
8	insurance_type	Health insurance of the patient	object	text	-	Mutuelle de Sante, RSSB, Private, None. We rea...	Assumed
9	service_department	Department that received the patient	object	text	-	7 departments	Assumed
10	appointment_type	How the visit was arranged	object	text	-	Walk-in, Scheduled, Follow-up, Referral	Assumed
11	arrival_hour	Hour of the day the patient arrived	int64	whole number	hour (24h clock)	0 to 23 accepted; observed 7 to 16	Assumed
12	day_of_week	Day of the week of the visit	object	text	-	Monday to Sunday	Assumed
13	distance_to_facility_km	Distance from the patient's home to the facility	float64	number	km	Above 0 and up to 150 accepted	Assumed
14	estimated_travel_time_min	Estimated time to travel to the facility	float64	number	minutes	Above 0 and up to 480 accepted	Assumed
15	referral	Whether the patient was referred from another ...	object	text	-	Yes, No	Assumed
16	previous_visits_12m	Number of visits by the patient in the previou...	float64	whole number	visits	0 to 100 accepted	Assumed
17	waiting_time_min	TARGET. Minutes waited between arrival and con...	float64	number	minutes	Above 0 and up to 720 accepted. Empty when the...	Assumed
18	consultation_duration_min	Length of the consultation	float64	number	minutes	Known only after the wait ends (leakage, see 3.7)	Assumed
19	attended	Whether the patient attended the visit	object	text	-	Yes, No	Assumed

What this does. Builds the Data Dictionary (one line per column), checks that it lists every column of the file exactly once, shows it and saves it to `docs/data_dictionary.csv`.

What we decided. Because the file came with no documentation, all meanings, units and allowed ranges are assumptions and are labelled "Assumed". The allowed ranges are used again in Stage 3 and Stage 4 to look for impossible values.

2.7 Source Log

Field	Entry
Source	File `rwanda_health_facility_visits.csv`. Source: NISR (National Institute of Statistics of Rwanda), as stated by the group. This has not been verified: no link, download record or licence text is on file.
Link	**TO BE COMPLETED BY THE GROUP**
Date accessed	**TO BE COMPLETED BY THE GROUP**
Licence	**Not on file - TO BE COMPLETED BY THE GROUP** (check the NISR terms of use for this dataset)
Who collected it	Stated source: NISR. The collection method (survey, facility register,

	other) is not documented.
File fingerprint	MD5 `fe54529b0294f43151a9f5b8815d01a7`, 186,300 bytes, 1,200 rows x 20 columns
Limitations	(1) The source is stated by the group as NISR but is not verified; the link, collection method and licence are not documented. (2) No documentation of column meanings; all meanings are assumed. (3) Only about 55 records per district, too few to study one district alone. (4) 168 records have no waiting time. (5) Some records are not logical (adults in Child Health, men in Maternal Health; measured in section 3.4). (6) The record codes run exactly from 1 to 1,200 and the facility codes follow a fixed pattern. These are signs that the file may have been generated and may not be a real extract of facility records. This is our observation, not a confirmed fact.
Risk of applying it to Rwanda	Until the origin is confirmed, results from this data must not be presented as facts about Rwandan health facilities. If the data is synthetic or comes from elsewhere, patterns in it may not hold in Rwanda.
Ethical issues - privacy	No names, phone numbers or national IDs. If the records are real, the mix of district, facility, date, age and sex could help to identify a person, so the file should not be published. `visit_id` and `facility_code` are removed from the final files.
Ethical issues - bias	Referral hospitals have very few records (about 3%), so they are weakly represented. Districts are missing (22 of 30 are present). Visits where the patient did not attend have no waiting time, so the prepared data describes only patients who stayed to be seen.

Stage 3 - EDA and data audit (PC 1.3)

Rule for this stage: nothing is changed. Every check reads `raw`. Where a check needs tidy text, it uses a temporary copy that is thrown away.

Code - cell 7

```
RAW_SHAPE_BEFORE_EDA = raw.shape
RAW_HASH_BEFORE_EDA = int(pd.util.hash_pandas_object(raw, index=True).sum())   # in-memory fingerprint of raw

raw.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 20 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   visit_id                   1200 non-null   object
 1   province                   1200 non-null   object
 2   district                   1200 non-null   object
 3   facility_code              1200 non-null   object
 4   facility_type              1190 non-null   object
 5   visit_date                 1192 non-null   object
 6   patient_age_years          1178 non-null   float64
 7   sex                        1200 non-null   object
 8   insurance_type             1161 non-null   object
 9   service_department         1177 non-null   object
 10  appointment_type           1185 non-null   object
 11  arrival_hour               1200 non-null   int64
 12  day_of_week                1200 non-null   object
 13  distance_to_facility_km    1165 non-null   float64
 14  estimated_travel_time_min  1169 non-null   float64
 15  referral                   1188 non-null   object
 16  previous_visits_12m        1182 non-null   float64
 17  waiting_time_min           1032 non-null   float64
 18  consultation_duration_min  1025 non-null   float64
 19  attended                   1186 non-null   object
dtypes: float64(6), int64(1), object(13)
memory usage: 187.6+ KB
```

What this does. Records the shape and an in-memory fingerprint of `raw` (so that we can prove at the end of Stage 3 that the audit changed nothing), then prints `info()`.

What we found. 1,200 entries. Seven columns are complete (`visit_id`, `province`, `district`, `facility_code`, `sex`, `arrival_hour`, `day_of_week`). The other columns have fewer than 1,200 non-null values, so they contain gaps.

Code - cell 8

```
display(raw.describe().T.round(2))
```

Output

	count	mean	std	min	25%	50%	75%	max
patient_age_years	1178.0	30.79	18.85	0.0	16.0	30.0	44.0	90.0
arrival_hour	1200.0	9.46	1.97	7.0	8.0	9.0	11.0	16.0
distance_to_facility_km	1165.0	5.90	4.14	0.2	2.8	5.0	7.9	25.3
estimated_travel_time_min	1169.0	27.40	18.62	5.0	13.0	24.0	38.0	117.0
previous_visits_12m	1182.0	2.20	1.52	0.0	1.0	2.0	3.0	8.0
waiting_time_min	1032.0	40.59	19.61	3.0	26.0	40.0	54.0	105.0
consultation_duration_min	1025.0	15.65	6.82	4.0	11.0	16.0	20.0	41.0

What this does. `describe()` gives the count, mean, spread, minimum, quartiles and maximum of each numerical column.

What we found. No column has a negative value. Ages run from 0 to 90 and arrival hours from 7 to 16. The mean waiting time is about 40.6 minutes (median 40, maximum 105). Distance and travel time have maximums far above their upper quartiles (25.3 km against 7.9 km; 117 minutes against 38 minutes), which points to a long right tail.

Code - cell 9

```
text_cols = raw.select_dtypes("object").columns.tolist()
for col in text_cols:
    n_unique = raw[col].nunique(dropna=True)
    print(f"\n--- {col}: {n_unique} different values ---")
    if n_unique <= 40:
        print(raw[col].value_counts(dropna=False).to_string())
```

```
    else:
        print("Too many to list. Examples:", raw[col].dropna().unique()[:5].tolist())
```

Output

```
--- visit_id: 1195 different values ---
Too many to list. Examples: ['RW-HLTH-00591', 'RW-HLTH-01167', 'RW-HLTH-01173', 'RW-HLTH-00551', 'RW-HLTH-00067']

--- province: 5 different values ---
province
Western        279
Southern       278
Northern       236
Eastern        235
Kigali City    172

--- district: 29 different values ---
district
Nyabihu       69
Burera        63
Musanze       62
Huye          61
Nyarugenge    61
Gisagara      59
Rulindo       59
Nyanza        59
Rubavu        56
Kicukiro      56
Gasabo        55
Karongi       53
Bugesera      53
Muhanga       53
Kayonza       50
Gicumbi       50
Rwamagana     49
Ngororero     49
Rusizi        49
Ruhango       45
Gatsibo       40
Nyagatare     39
Ngororero      3
Nyagatare      2
Bugesera       1
Muhanga        1
Gatsibo        1
Rulindo        1
Burera         1

--- facility_code: 146 different values ---
Too many to list. Examples: ['DH-MUS-03', 'HC-NYA-02', 'HC-KAR-01', 'HC-GIC-02', 'HC-MUS-01']

--- facility_type: 3 different values ---
facility_type
Health Centre        818
District Hospital    337
Referral Hospital     35
NaN                   10

--- visit_date: 579 different values ---
Too many to list. Examples: ['2025-02-11', '2025-01-17', '2024-02-13', '2025-09-29', '2025-06-08']

--- sex: 6 different values ---
sex
Male      570
Female    559
male       18
FEMALE     18
MALE       18
female     17

--- insurance_type: 7 different values ---
insurance_type
Mutuelle de Santé     802
RSSB                  153
```

[Output shortened here for the report; the full output is in the notebook.]

What this does. Lists the different values of every text column with their counts. Columns with more than 40 different values (`visit_id`, `facility_code`, `visit_date`) show five examples only.

What we found.

- `sex`, `referral` and `attended` each show six spellings for two real values (for example `Male`, `male`, `MALE`).
- `district` shows 29 values for 22 real districts; seven names appear twice because of a space at the end (for example `Ngororero` and `Ngororero `).
- `insurance_type` shows `None` 132 times and `None ` (with a space) 5 times, as well as `RSSB ` and `Mutuelle de Santé ` with a space.
- `visit_id` has 1,195 different values in 1,200 rows, so five codes are repeated. `facility_code` has 146 values as written.
- `province`, `facility_type`, `service_department`, `appointment_type` and `day_of_week` have consistent spellings.

3.1 Missing values, including disguised ones

Code - cell 10

```
DISGUISED_TOKENS = ["?", "na", "n/a", "nan", "null", "none", "unknown", "missing", "-", "-999", "999"]
SENTINEL_NUMBERS = [-999, -99, 999, 9999]
ZERO_IMPOSSIBLE = ["distance_to_facility_km", "estimated_travel_time_min", "waiting_time_min",
"consultation_duration_min"]

rows = []
for col in raw.columns:
    s = raw[col]
    is_num = pd.api.types.is_numeric_dtype(s)
    tokens = 0 if is_num else int(s.dropna().str.strip().str.lower().isin(DISGUISED_TOKENS).sum())
    sentinels = int(s.isin(SENTINEL_NUMBERS).sum()) if is_num else 0
    zeros = int((s == 0).sum()) if col in ZERO_IMPOSSIBLE else 0
    rows.append([col, int(s.isna().sum()), round(100 * s.isna().mean(), 1), tokens, sentinels, zeros])
missing_table = pd.DataFrame(rows, columns=["Column", "Empty cells", "Empty %", "Disguised text tokens",
                                            "Sentinel numbers (-999 etc.)", "Zeros where zero is impossible"])
display(missing_table)
print("Zero ages (allowed: infants under one year):", int((raw["patient_age_years"] == 0).sum()))
print("Zero previous visits (allowed: first visit) :", int((raw["previous_visits_12m"] == 0).sum()))
```

Output

	Column	Empty cells	Empty %	Disguised text tokens	Sentinel numbers (-999 etc.)	Zeros where zero is impossible
0	visit_id	0	0.0	0	0	0
1	province	0	0.0	0	0	0
2	district	0	0.0	0	0	0
3	facility_code	0	0.0	0	0	0
4	facility_type	10	0.8	0	0	0
5	visit_date	8	0.7	0	0	0
6	patient_age_years	22	1.8	0	0	0
7	sex	0	0.0	0	0	0
8	insurance_type	39	3.2	137	0	0
9	service_department	23	1.9	0	0	0
10	appointment_type	15	1.2	0	0	0
11	arrival_hour	0	0.0	0	0	0
12	day_of_week	0	0.0	0	0	0
13	distance_to_facility_km	35	2.9	0	0	0
14	estimated_travel_time_min	31	2.6	0	0	0
15	referral	12	1.0	0	0	0
16	previous_visits_12m	18	1.5	0	0	0
17	waiting_time_min	168	14.0	0	0	0
18	consultation_duration_min	175	14.6	0	0	0
19	attended	14	1.2	0	0	0

```
Zero ages (allowed: infants under one year): 72
Zero previous visits (allowed: first visit) : 138
```

What this does. Counts, for every column, the empty cells and three kinds of disguised missing value: text tokens such as "?", "NA", "unknown" or "None"; sentinel numbers such as -999; and zeros in columns where zero is not possible (a distance, travel time, waiting time or consultation of 0).

What we found.

- 14 of the 20 columns have empty cells. The largest gaps are `consultation_duration_min` (175, 14.6%), `waiting_time_min` (168, 14.0%) and `insurance_type` (39, 3.2%).
- The only disguised token is the word "None" in `insurance_type` (137 records).
- No sentinel numbers and no impossible zeros were found (checked: none found).

- 72 records have age 0 and 138 have 0 previous visits. Both are possible values, so they are not treated as missing.

What we decided (assumption). We read `None` in `insurance_type` as a real answer, "the patient has no insurance", and not as a missing value. Reason: it is a frequent, consistent label (137 records) next to 39 truly empty cells, and uninsured patients are a real group. This cannot be confirmed without the data owner, so it is recorded as an assumption.

Code - cell 11

```
missing_counts = raw.isna().sum().sort_values(ascending=False)
missing_counts = missing_counts[missing_counts > 0]
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.barh(missing_counts.index[::-1], missing_counts.values[::-1], color="steelblue")
for i, v in enumerate(missing_counts.values[::-1]):
    ax.text(v + 1, i, str(v), va="center", fontsize=9)
ax.set_xlabel("Number of empty cells (out of 1,200 records)")
ax.set_title("Figure 1 - Empty cells per column")
plt.tight_layout()
plt.show()
```

Output

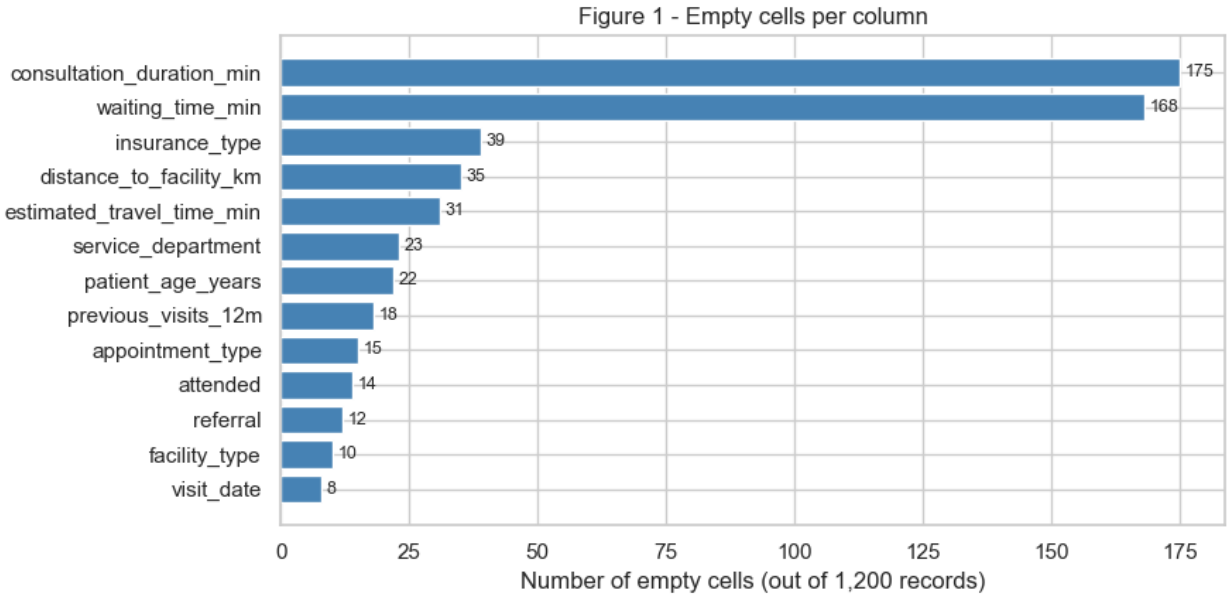


Figure 1 (bar chart) - interpretation. Two columns stand far above the rest: `consultation_duration_min` (175 empty) and `waiting_time_min` (168 empty), followed by `insurance_type` (39); every other column has 35 or fewer empty cells.

3.2 Duplicates

Code - cell 12

```
def tidy_view(df):
    """Temporary copy with spaces removed and lower-case text. Used for counting only."""
    view = df.copy()
    for c in view.select_dtypes("object").columns:
        view[c] = view[c].str.strip().str.lower()
    return view


non_id_cols = [c for c in raw.columns if c != "visit_id"]
view = tidy_view(raw)

full_dups = int(raw.duplicated().sum())
full_dups_tidy = int(view.duplicated().sum())
repeated_ids = int(raw["visit_id"].duplicated().sum())
ids_with_different_content = repeated_ids - int(view.duplicated().sum())
same_rows = view[view.duplicated(subset=non_id_cols, keep=False)]
same_values_other_id = int((same_rows.groupby(non_id_cols, dropna=False)["visit_id"].nunique() > 1).sum())

duplicate_table = pd.DataFrame([
    ["Full duplicate rows (exact copy)", full_dups],
    ["Full duplicate rows after tidying text (spaces, upper/lower case)", full_dups_tidy],
    ["Repeated visit_id values", repeated_ids],
```

```
    ["Repeated visit_id with different content", ids_with_different_content],
    ["Same values under a different visit_id", same_values_other_id],
], columns=["Check", "Records"])
display(duplicate_table)
display(raw[raw["visit_id"].duplicated(keep=False)].sort_values("visit_id").iloc[:, :8])
```

Output

	Check	Records
0	Full duplicate rows (exact copy)	5
1	Full duplicate rows after tidying text (spaces...	5
2	Repeated visit_id values	5
3	Repeated visit_id with different content	0
4	Same values under a different visit_id	0

	visit_id	province	district	facility_code	facility_type	visit_date	patient_age_years	sex
110	RW-HLTH-00165	Kigali City	Nyarugenge	DH-NYA-01	District Hospital	2024-09-29	28.0	Male
172	RW-HLTH-00165	Kigali City	Nyarugenge	DH-NYA-01	District Hospital	2024-09-29	28.0	Male
176	RW-HLTH-00325	Eastern	Bugesera	HC-BUG-03	Health Centre	2024-05-22	0.0	Male
288	RW-HLTH-00325	Eastern	Bugesera	HC-BUG-03	Health Centre	2024-05-22	0.0	Male
525	RW-HLTH-00599	Western	Nyabihu	HC-NYA-03	Health Centre	2025-05-30	12.0	Male
1021	RW-HLTH-00599	Western	Nyabihu	HC-NYA-03	Health Centre	2025-05-30	12.0	Male
131	RW-HLTH-00993	Kigali City	Kicukiro	HC-KIC-02	Health Centre	2024-02-24	32.0	Female
611	RW-HLTH-00993	Kigali City	Kicukiro	HC-KIC-02	Health Centre	2024-02-24	32.0	Female
100	RW-HLTH-01138	Northern	Rulindo	DH-RUL-01	District Hospital	2024-07-28	50.0	Male
924	RW-HLTH-01138	Northern	Rulindo	DH-RUL-01	District Hospital	2024-07-28	50.0	Male

What this does. Looks for three kinds of duplicate: rows that are exact copies, `visit_id` codes that appear more than once, and rows with the same values but a different `visit_id`. The checks are repeated on a tidy view so that `Male` and `male` do not hide a duplicate.

What we found. Five rows are exact copies of another row. They are the same five repeated `visit_id` codes, so no repeated code has different content. No record appears under two different codes (checked: none found).

What we decided. In Stage 4 we keep the first copy of each duplicated row and remove the other five.

3.3 Inconsistent spellings and formats

Code - cell 13

```
rows = []
for col in text_cols:
    s = raw[col].dropna()
    stripped = s.str.strip()
    rows.append([col, s.nunique(), stripped.str.lower().nunique(),
                 int((s != stripped).sum()),
                 int((stripped != stripped.str.title()).sum()) if col in ["sex", "referral", "attended"] else 0])
spelling_table = pd.DataFrame(rows, columns=["Column", "Different values as written", "Different values after tidying",
                                             "Records with extra spaces", "Records not in 'Title' case"])
display(spelling_table)

date_ok = raw["visit_date"].dropna().str.match(r"^\d{4}-\d{2}-\d{2}$")
parsed = pd.to_datetime(raw["visit_date"], format="%Y-%m-%d", errors="coerce")
print("Dates not written as YYYY-MM-DD      :", int((~date_ok).sum()))
print("Dates that are not real calendar days:", int((parsed.isna() & raw["visit_date"].notna()).sum()))
print("Date range                           :", parsed.min().date(), "to", parsed.max().date())
print("day_of_week different from the date  :", int((parsed.dt.day_name() !=
raw["day_of_week"])[parsed.notna()].sum()))
print("Ages that are not whole numbers      :", int((raw["patient_age_years"].dropna() % 1 != 0).sum()))
```

Output

	Column	Different values as written	Different values after tidying	Records with extra spaces	Records not in 'Title' case
0	visit_id	1195	1195	0	0
1	province	5	5	0	0
2	district	29	22	10	0

3	facility_code	146	138	10	0
4	facility_type	3	3	0	0
5	visit_date	579	579	0	0
6	sex	6	2	0	71
7	insurance_type	7	4	10	0
8	service_department	7	7	0	0
9	appointment_type	4	4	0	0
10	day_of_week	7	7	0	0
11	referral	6	2	0	71
12	attended	6	2	0	70

```
Dates not written as YYYY-MM-DD      : 0
Dates that are not real calendar days: 0
Date range                           : 2024-01-02 to 2025-12-30
day_of_week different from the date  : 0
Ages that are not whole numbers      : 0
```

What this does. For each text column, compares the number of different values as written with the number after removing spaces and ignoring upper/lower case. It also checks the date format, whether each date is a real day, whether day_of_week agrees with the date, and whether ages are whole numbers.

What we found.

- Extra spaces at the end of the value: 10 records each in `district`, `facility_code` and `insurance_type`.
- Mixed upper/lower case: 71 records in `sex`, 71 in `referral` and 70 in `attended`.
- Dates: all are written as YYYY-MM-DD, all are real days, and `day_of_week` always agrees with the date (checked: none found). Units look consistent: no column mixes formats.

What we decided. In Stage 4 we remove the extra spaces and write `sex`, `referral` and `attended` in one form (`Male`, `Yes`).

3.4 Impossible and illogical values

Code - cell 14

```
VALID_RANGES = {                                # assumptions, also listed in the Data Dictionary
    "patient_age_years": (0, 110),
    "arrival_hour": (0, 23),
    "distance_to_facility_km": (0.01, 150),
    "estimated_travel_time_min": (1, 480),
    "previous_visits_12m": (0, 100),
    "waiting_time_min": (0.01, 720),
    "consultation_duration_min": (0.01, 480),
}
rows = []
for col, (low, high) in VALID_RANGES.items():
    s = raw[col]
    rows.append([col, low, high, s.min(), s.max(), int(((s < low) | (s > high)).sum())])
range_table = pd.DataFrame(rows, columns=["Column", "Lowest allowed", "Highest allowed", "Lowest found",
                                          "Highest found", "Records outside the range"])
display(range_table)

view = tidy_view(raw)
speed = raw["distance_to_facility_km"] / (raw["estimated_travel_time_min"] / 60)
prefix_type = {"hc": "health centre", "dh": "district hospital", "rh": "referral hospital"}
logic_table = pd.DataFrame([
    ["Travel speed below 1 km/h or above 80 km/h", int(((speed < 1) | (speed > 80)).sum())],
    ["District listed under more than one province", int((view.groupby("district")["province"].nunique() > 1).sum())],
    ["facility_type different from the facility_code prefix (HC/DH/RH)",
     int((view["facility_code"].str[:2].map(prefix_type) !=
view["facility_type"])[view["facility_type"].notna()].sum())],
    ["Facility codes used in more than one district", int((view.groupby("facility_code")["district"].nunique() >
1).sum())],
    ["Did not attend but has a waiting time", int(((view["attended"] == "no") & raw[TARGET].notna()).sum())],
    ["Attended but has no waiting time", int(((view["attended"] == "yes") & raw[TARGET].isna()).sum())],
    ["'attended' empty but a waiting time is recorded", int((view["attended"].isna() & raw[TARGET].notna()).sum())],
    ["Child Health visit with a patient aged 18 or more",
     int(((view["service_department"] == "child health") & (raw["patient_age_years"] >= 18)).sum())],
    ["Maternal Health visit with a male patient",
     int(((view["service_department"] == "maternal health") & (view["sex"] == "male")).sum())],
], columns=["Logic check", "Records"])
display(logic_table)
print("Travel speed (km/h): lowest", round(speed.min(), 1), "| median", round(speed.median(), 1), "| highest",
round(speed.max(), 1))
```

Output

	Column	Lowest allowed	Highest allowed	Lowest found	Highest found	Records outside the range
0	patient_age_years	0.00	110	0.0	90.0	0
1	arrival_hour	0.00	23	7.0	16.0	0
2	distance_to_facility_km	0.01	150	0.2	25.3	0
3	estimated_travel_time_min	1.00	480	5.0	117.0	0
4	previous_visits_12m	0.00	100	0.0	8.0	0
5	waiting_time_min	0.01	720	3.0	105.0	0
6	consultation_duration_min	0.01	480	4.0	41.0	0

	Logic check	Records
0	Travel speed below 1 km/h or above 80 km/h	0
1	District listed under more than one province	0
2	facility_type different from the facility_code...	0
3	Facility codes used in more than one district	8
4	Did not attend but has a waiting time	0
5	Attended but has no waiting time	24
6	'attended' empty but a waiting time is recorded	12
7	Child Health visit with a patient aged 18 or more	105
8	Maternal Health visit with a male patient	77

```
Travel speed (km/h): lowest 1.7 | median 12.7 | highest 68.4
```

What this does. First, compares each numerical column with the allowed range from the Data Dictionary. Second, runs logic checks between columns: travel speed (distance divided by travel time), district against province, facility type against the facility code, attendance against waiting time, and department against age and sex.

What we found.

- No value is outside its allowed range (checked: none found). Travel speeds run from 1.7 to 68.4 km/h, which is possible on foot or by vehicle.
- Each district belongs to one province, and `facility_type` always agrees with the code prefix (HC, DH, RH).
- 8 facility codes are used in more than one district (for example `HC-NYA-02` appears for Nyabihu, Nyanza, Nyagatare and Nyarugenge), so `facility_code` alone does not identify a facility.
- No "did not attend" record has a waiting time. 24 attended records have no waiting time. 12 records have an empty `attended` but do have a waiting time.
- **Illogical records:** 105 Child Health visits have a patient aged 18 or more, and 77 Maternal Health visits have a male patient.

What we decided.

- The age limit of 18 for Child Health and "Maternal Health patients are female" are our assumptions.
- We cannot tell which column is wrong in the illogical records (the department, the age or the sex), and they are too many to be typing errors. Changing them would mean inventing data. We keep them, report them in the Data Quality Report and list them as a limitation in the Data Card.
- A record with a waiting time is treated as an attended visit, even if `attended` is empty.

3.5 Outliers: box plots and the IQR rule

Code - cell 15

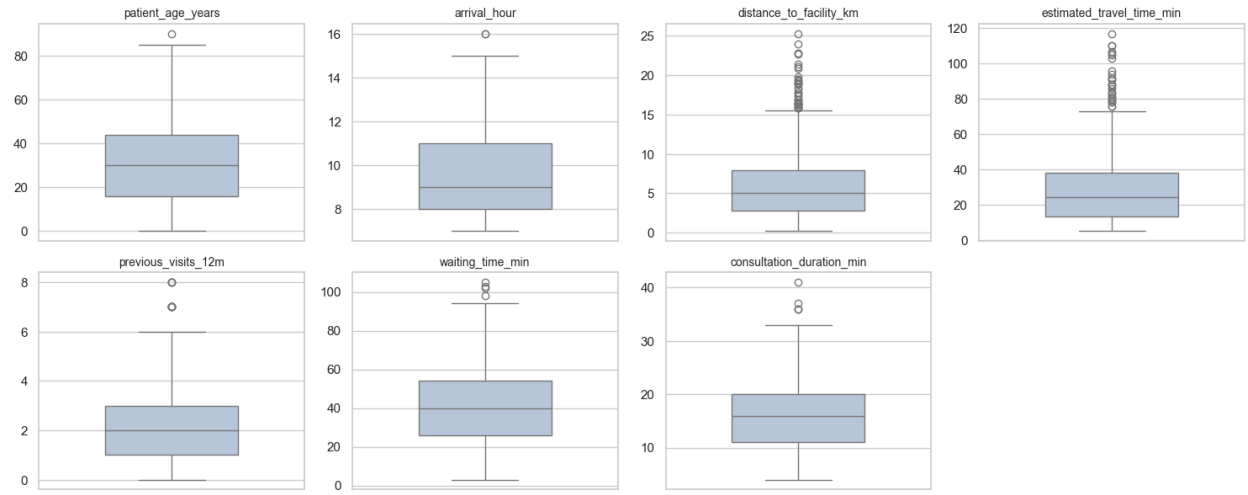
```
num_cols = raw.select_dtypes("number").columns.tolist()
fig, axes = plt.subplots(2, 4, figsize=(15, 6.5))
for ax, col in zip(axes.ravel(), num_cols):
    sns.boxplot(y=raw[col], ax=ax, color="lightsteelblue", width=0.5)
    ax.set_title(col, fontsize=10)
    ax.set_ylabel("")
axes.ravel()[-1].axis("off")
fig.suptitle("Figure 2 - Box plots of the numerical columns")
plt.tight_layout()
plt.show()

rows = []
for col in num_cols:
    q1, q3 = raw[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
```

```
    rows.append([col, q1, q3, iqr, round(low, 2), round(high, 2),
                 int((raw[col] < low).sum()), int((raw[col] > high).sum()), raw[col].max()])
outlier_table = pd.DataFrame(rows, columns=["Column", "Q1", "Q3", "IQR", "Lower limit", "Upper limit",
                                            "Below lower limit", "Above upper limit", "Highest value"])
display(outlier_table)
```

Output

Figure 2 - Box plots of the numerical columns



	Column	Q1	Q3	IQR	Lower limit	Upper limit	Below lower limit	Above upper limit	Highest value
0	patient_age_years	16.0	44.0	28.0	-26.00	86.00	0	1	90.0
1	arrival_hour	8.0	11.0	3.0	3.50	15.50	0	2	16.0
2	distance_to_facility_km	2.8	7.9	5.1	-4.85	15.55	0	37	25.3
3	estimated_travel_time_min	13.0	38.0	25.0	-24.50	75.50	0	27	117.0
4	previous_visits_12m	1.0	3.0	2.0	-2.00	6.00	0	12	8.0
5	waiting_time_min	26.0	54.0	28.0	-16.00	96.00	0	4	105.0
6	consultation_duration_min	11.0	20.0	9.0	-2.50	33.50	0	5	41.0

What this does. Draws a box plot of each numerical column and applies the IQR rule: a value is flagged as an outlier if it is below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR. The table gives the limits and the counts.

Figure 2 (box plots) - interpretation. All flagged points lie above the upper limit and none below; `distance_to_facility_km` (37 points) and `estimated_travel_time_min` (27 points) have the longest upper tails, while the target has only 4 flagged points.

What we found. Flagged above the upper limit: distance 37, travel time 27, previous visits 12, consultation 5, waiting time 4, arrival hour 2, age 1. All of them are possible values: the furthest patient lives 25.3 km away, the longest travel is 117 minutes, the longest wait is 105 minutes and the oldest patient is 90.

What we decided. The decision per column is made in Stage 4 (section 4.6). Our proposal: keep every value, because none is impossible, and reduce the long tails of distance and travel time with a transformation in Stage 5.

3.6 Target study

Code - cell 16

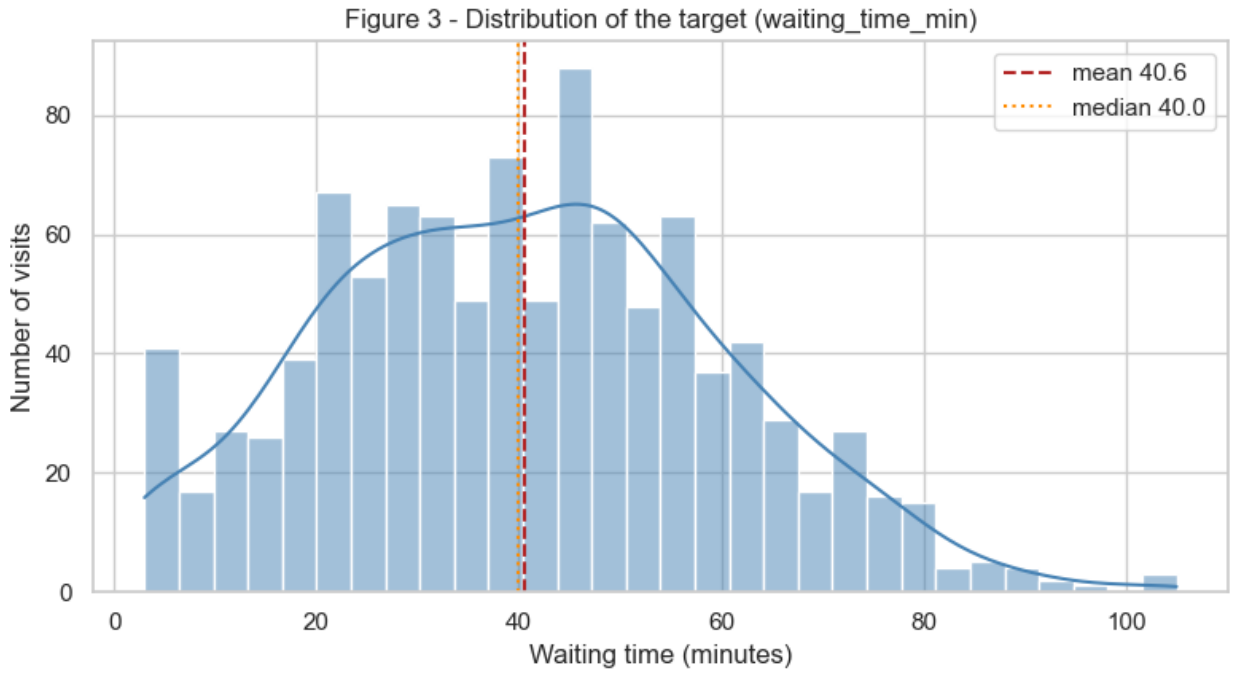
```
target = raw[TARGET]
print(target.describe().round(2).to_string())
print("Skewness:", round(target.skew(), 2))
print("Empty   :", int(target.isna().sum()), "of", len(raw))
print()
print("Empty target by attendance (tidy view):")
print(pd.crosstab(tidy_view(raw)["attended"].fillna("(empty)"), target.isna().map({True: "target empty", False: "target
present"})))

fig, ax = plt.subplots(figsize=(8, 4.5))
sns.histplot(target.dropna(), bins=30, kde=True, color="steelblue", ax=ax)
ax.axvline(target.mean(), color="firebrick", linestyle="--", label=f"mean {target.mean():.1f}")
ax.axvline(target.median(), color="darkorange", linestyle=":", label=f"median {target.median():.1f}")
ax.set_xlabel("Waiting time (minutes)")
ax.set_ylabel("Number of visits")
ax.set_title("Figure 3 - Distribution of the target (waiting_time_min)")
ax.legend()
plt.tight_layout()
plt.show()
```

Output

```
count    1032.00
mean       40.59
std        19.61
min         3.00
25%        26.00
50%        40.00
75%        54.00
max       105.00
Skewness: 0.22
Empty   : 168 of 1200

Empty target by attendance (tidy view):
waiting_time_min  target empty  target present
attended
(empty)                      2              12
no                         142               0
yes                         24            1020
```



What this does. Describes the target, measures its skewness, counts the empty values and shows how the empty values relate to attendance. Figure 3 is the histogram of the target.

Figure 3 (histogram) - interpretation. The waiting time is spread widely around 40 minutes (mean 40.6, median 40.0) from 3 to 105 minutes, with only a mild right tail (skewness 0.22).

What we found. 168 records have no waiting time: all 142 records where the patient did not attend, 24 attended records and 2 records with an empty `attended`. The target is a number, so there is no class balance to study; the question of imbalance is taken up in section 4.12.

What we decided. Records without a target cannot be used for supervised learning and will be removed in Stage 4. Because the target is nearly symmetric, it will not be transformed, and it is never scaled.

3.7 Relationships (charts 4 to 7)

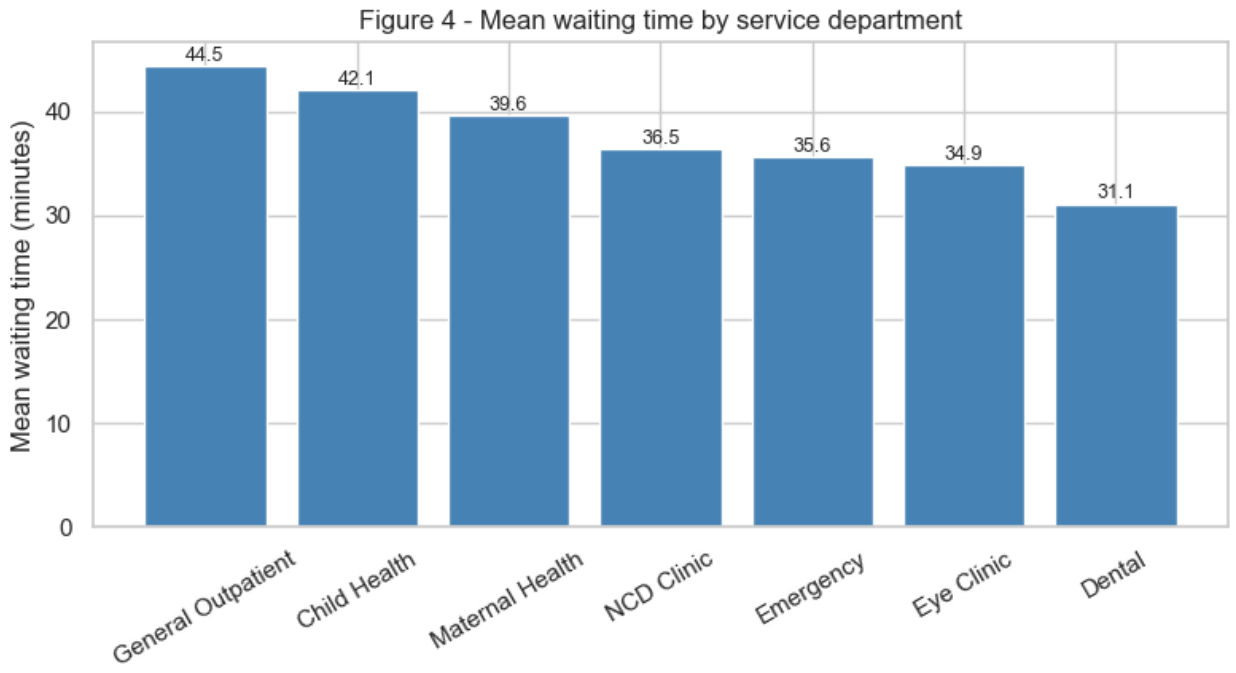
Code - cell 17

```
view = raw.copy()
for c in ["facility_type", "service_department"]:
    view[c] = view[c].str.strip()

dept_mean = view.groupby("service_department")[TARGET].mean().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.bar(dept_mean.index, dept_mean.values, color="steelblue")
for i, v in enumerate(dept_mean.values):
    ax.text(i, v + 0.5, f"{v:.1f}", ha="center", fontsize=9)
ax.set_ylabel("Mean waiting time (minutes)")
ax.set_title("Figure 4 - Mean waiting time by service department")
ax.tick_params(axis="x", rotation=30)
```

```
plt.tight_layout()
plt.show()
print(view.groupby("service_department")[TARGET].agg(["count", "mean", "median"]).round(1).sort_values("mean",
ascending=False))
```

Output



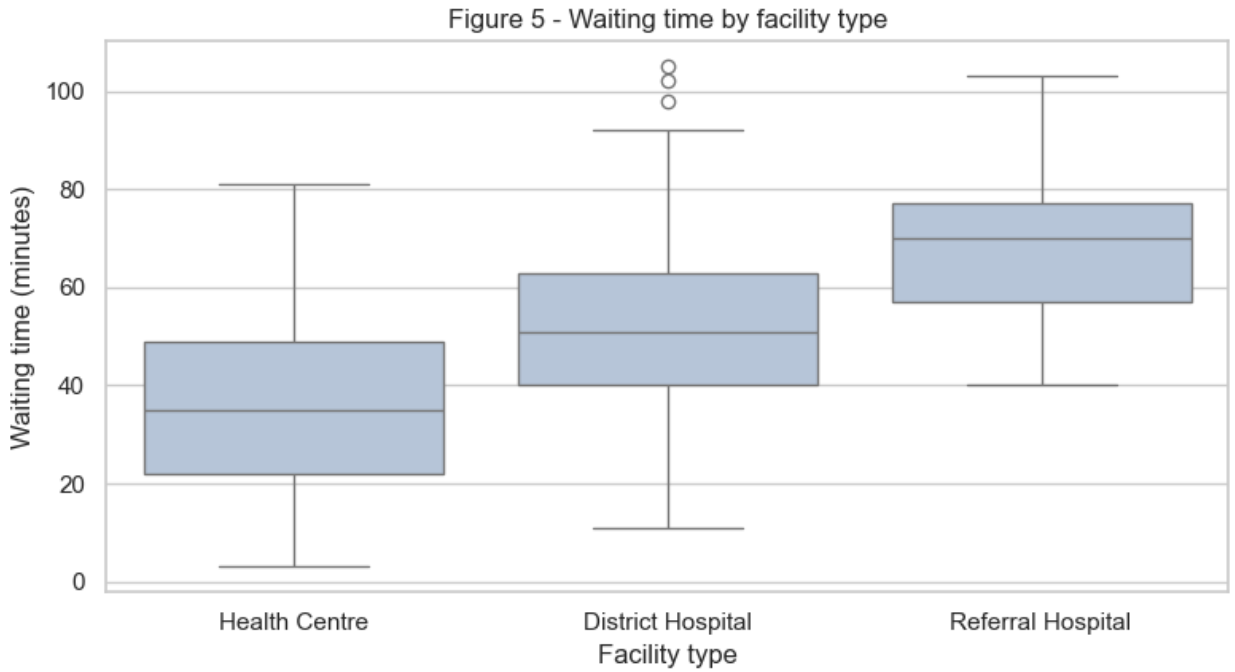
service_department	count	mean	median
General Outpatient	431	44.5	45.0
Child Health	134	42.1	42.0
Maternal Health	126	39.6	41.0
NCD Clinic	126	36.5	34.0
Emergency	84	35.6	37.0
Eye Clinic	59	34.9	32.0
Dental	51	31.1	31.0

Figure 4 (bar chart) - interpretation. General Outpatient has the longest mean wait (44.5 minutes) and Dental the shortest (31.1 minutes), a gap of about 13 minutes between departments (BQ1).

Code - cell 18

```
order = ["Health Centre", "District Hospital", "Referral Hospital"]
fig, ax = plt.subplots(figsize=(8, 4.5))
sns.boxplot(data=view, x="facility_type", y=TARGET, order=order, color="lightsteelblue", ax=ax)
ax.set_xlabel("Facility type")
ax.set_ylabel("Waiting time (minutes)")
ax.set_title("Figure 5 - Waiting time by facility type")
plt.tight_layout()
plt.show()
print(view.groupby("facility_type")[TARGET].agg(["count", "mean", "median"]).round(1).reindex(order))
```

Output



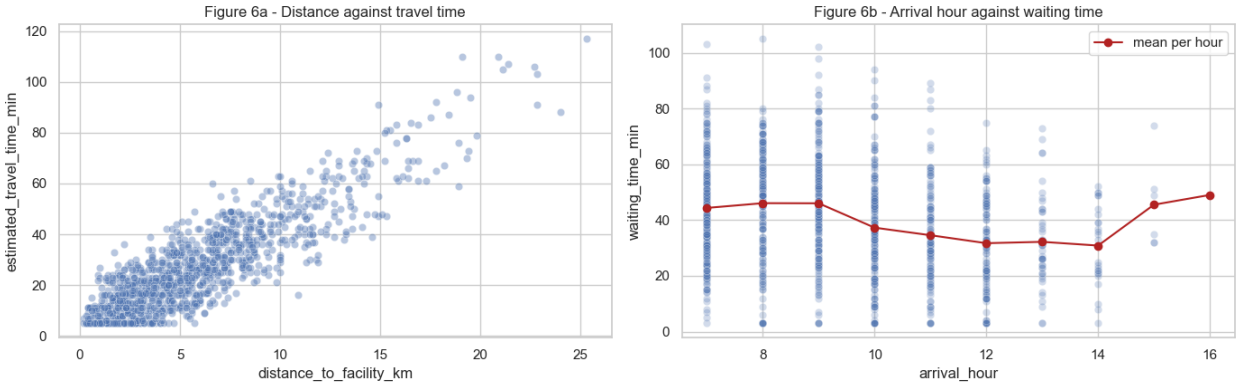

```
                  count  mean  median
facility_type
Health Centre       710  35.3    35.0
District Hospital   285  51.0    51.0
Referral Hospital    29  69.0    70.0
```

Figure 5 (box plot) - interpretation. Waiting time rises with the level of the facility: the median is 35 minutes at health centres, 51 at district hospitals and 70 at referral hospitals, although referral hospitals have only 29 records with a waiting time (BQ1).

Code - cell 19

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
sns.scatterplot(data=raw, x="distance_to_facility_km", y="estimated_travel_time_min", alpha=0.4, ax=axes[0])
axes[0].set_title("Figure 6a - Distance against travel time")
sns.scatterplot(data=raw, x="arrival_hour", y=TARGET, alpha=0.25, ax=axes[1])
hour_mean = raw.groupby("arrival_hour")[TARGET].mean()
axes[1].plot(hour_mean.index, hour_mean.values, color="firebrick", marker="o", label="mean per hour")
axes[1].set_title("Figure 6b - Arrival hour against waiting time")
axes[1].legend()
plt.tight_layout()
plt.show()
print("Correlation distance vs travel time:",
round(raw["distance_to_facility_km"].corr(raw["estimated_travel_time_min"]), 2))
print(raw.groupby("arrival_hour")[TARGET].agg(["count", "mean"]).round(1).T)
```

Output



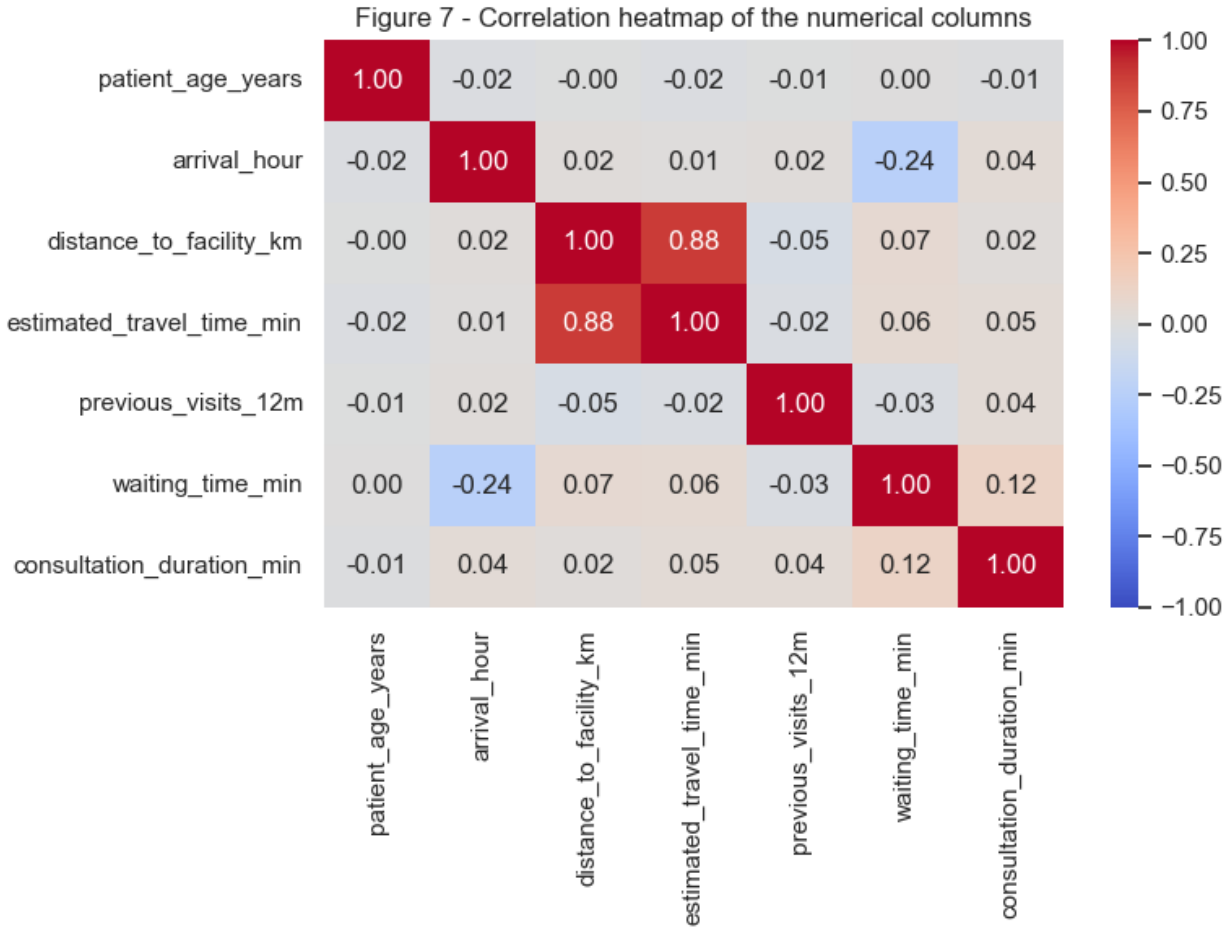

```
Correlation distance vs travel time: 0.88
arrival_hour     7      8      9      10     11     12     13     14     15     16
count         222.0  160.0  190.0  163.0  121.0  100.0  43.0   26.0   6.0    1.0
mean           44.4   46.0   46.0   37.3   34.6   31.7   32.2   30.8   45.5   49.0
```

Figure 6 (scatter plots) - interpretation. (a) Travel time rises almost in a straight line with distance (correlation 0.88), so the two columns carry much of the same information; (b) patients arriving at 7, 8 or 9 wait about 44 to 46 minutes on average, while those arriving from 10 to 14 wait about 31 to 37 minutes (BQ2). The averages at 15 and 16 rest on only 7 visits and should not be trusted.

Code - cell 20

```
corr = raw[num_cols].corr()
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1, ax=ax)
ax.set_title("Figure 7 - Correlation heatmap of the numerical columns")
plt.tight_layout()
plt.show()
print(corr[TARGET].round(3).sort_values().to_string())
```

Output




```
arrival_hour               -0.243
previous_visits_12m        -0.027
patient_age_years           0.001
estimated_travel_time_min   0.061
distance_to_facility_km     0.069
consultation_duration_min   0.124
waiting_time_min            1.000
```

Figure 7 (correlation heatmap) - interpretation. The only strong pair is distance and travel time (0.88); the target has a weak negative correlation with arrival hour (-0.24), a weak positive one with consultation duration (0.12) and almost none with age, distance, travel time and previous visits.

What we found. Straight-line correlations with the target are weak. The clearest patterns are by category (facility type and department, Figures 4 and 5) and by arrival hour.

Note on fairness to the test set. These charts describe the whole file before any split, as the task asks for the audit. They are used to understand data quality. Every feature decision in Stage 5 is taken again from the training set only.

3.8 Leakage check

Code - cell 21

```
view = tidy_view(raw)
leakage_table = pd.DataFrame([
    ["consultation_duration_min", "After the wait has ended", "Yes - leakage", "Drop"],
    ["attended", "Only once the patient has (or has not) been seen", "Yes - outcome status", "Drop"],
    ["waiting_time_min", "This is the target", "-", "Keep as target"],
    ["visit_id", "Before (record code)", "No, but it is an identifier", "Drop from features"],
    ["facility_code", "Before (facility code)", "No, but it is an identifier-like code", "Drop from features"],
    ["All other columns", "At or before arrival", "No", "Keep"],
], columns=["Column", "When is it known?", "Known only after the outcome?", "Proposed action"])
display(leakage_table)

print("consultation_duration_min empty when the patient did not attend:",
      int(((view["attended"] == "no") & raw["consultation_duration_min"].isna()).sum()), "of",
      int((view["attended"] == "no").sum()))
print("attended values among records that have a waiting time:",
      view.loc[raw[TARGET].notna(), "attended"].value_counts(dropna=False).to_dict())
```

Output

	Column	When is it known?	Known only after the outcome?	Proposed action
0	consultation_duration_min	After the wait has ended	Yes - leakage	Drop
1	attended	Only once the patient has (or has not) been seen	Yes - outcome status	Drop
2	waiting_time_min	This is the target	-	Keep as target
3	visit_id	Before (record code)	No, but it is an identifier	Drop from features
4	facility_code	Before (facility code)	No, but it is an identifier-like code	Drop from features
5	All other columns	At or before arrival	No	Keep

```
consultation_duration_min empty when the patient did not attend: 142 of 142
attended values among records that have a waiting time: {'yes': 1020, nan: 12}
```

What this does. Asks of each column: "would we know this value at the moment the patient arrives, before the wait is over?" A column that is known only afterwards would leak the answer into the features.

What we found.

- `consultation_duration_min` is measured after the wait has ended. It is empty for all 142 visits where the patient did not attend, which confirms it is recorded only after the outcome.
- `attended` is `yes` or empty for every record that has a waiting time, so it carries no information for our target and describes the outcome of the visit.
- All other columns (facility, date, arrival hour, patient details, distance, appointment type, referral, previous visits) are known at or before arrival. That `previous_visits_12m` counts only earlier visits is an assumption from its name.

What we decided. Drop `consultation_duration_min` and `attended` in Stage 4. Drop `visit_id` and `facility_code` as identifiers.

3.9 Data Quality Report

Code - cell 22

```
view = tidy_view(raw)
n_case = {c: int((raw[c].dropna().str.strip() != raw[c].dropna().str.strip().str.title()).sum()) for c in ["sex",
"referral", "attended"]}
n_space = {c: int((raw[c].dropna() != raw[c].dropna().str.strip()).sum()) for c in text_cols}
n_space = {c: n for c, n in n_space.items() if n > 0}
feature_gap_cols = [c for c in raw.columns if raw[c].isna().sum() > 0 and c not in [TARGET,
"consultation_duration_min", "attended"]]

quality_report = pd.DataFrame([
    ["Target is empty", TARGET, int(raw[TARGET].isna().sum()), "High", "Remove the records (a record without a target
cannot be used)"],
    ["Leakage: known only after the outcome", "consultation_duration_min, attended", len(raw), "High", "Drop both
columns"],
    ["Empty cells in feature columns", ", ".join(feature_gap_cols),
     int(raw[feature_gap_cols].isna().any(axis=1).sum()), "Medium",
     "Impute after the split with values learned from the training set (facility_type is rebuilt from the code
prefix)"],
    ["Disguised value: the word 'None'", "insurance_type", int((view["insurance_type"] == "none").sum()), "Medium",
     "Keep as the category 'None' = no insurance (assumption)"],
    ["Full duplicate rows / repeated visit_id", "all / visit_id", full_dups, "Medium", "Keep the first copy, remove the
```

```
rest"],
    ["Same values under a different visit_id", "all", same_values_other_id, "-", "Checked: none found"],
    ["Mixed upper/lower case", ", ".join(n_case), sum(n_case.values()), "Medium", "Write in one form (Title case)"],
    ["Extra spaces at the end of text", ", ".join(n_space), sum(n_space.values()), "Medium", "Remove the spaces"],
    ["Wrong data type", "visit_date (text)", len(raw), "Low", "Convert to a date"],
    ["Inconsistent date formats or units", "visit_date, numerical columns", 0, "-", "Checked: none found"],
    ["day_of_week different from visit_date", "day_of_week, visit_date", 0, "-", "Checked: none found"],
    ["Values outside the allowed range", ", ".join(VALID_RANGES), int(range_table["Records outside the range"].sum()),
"-", "Checked: none found"],
    ["Sentinel numbers or impossible zeros", "numerical columns",
     int(missing_table["Sentinel numbers (-999 etc.)"].sum() + missing_table["Zeros where zero is impossible"].sum()),
"-", "Checked: none found"],
    ["Illogical: Child Health patient aged 18+", "service_department, patient_age_years",
     int(logic_table.loc[7, "Records"]), "Medium", "Keep and report as a limitation (cannot tell which column is
wrong)"],
    ["Illogical: male patient in Maternal Health", "service_department, sex",
     int(logic_table.loc[8, "Records"]), "Medium", "Keep and report as a limitation (cannot tell which column is
wrong)"],
    ["Facility code used in several districts", "facility_code", int(logic_table.loc[3, "Records"]), "Low", "Drop the
column (identifier-like)"],
    ["Outliers by the IQR rule (all are possible values)", "distance, travel time, previous visits, consultation,
waiting time, arrival hour, age",
     int(outlier_table["Above upper limit"].sum() + outlier_table["Below lower limit"].sum()), "Low",
     "Keep; reduce the tails of skewed columns by transformation in Stage 5"],
    ["Source stated as NISR but not verified; link, licence and column meanings not documented", "whole file",
len(raw), "High", "Record in the Source Log; the group must add the link, date accessed and licence"],
], columns=["Problem", "Columns affected", "Records affected", "Severity", "Proposed treatment"])
quality_report.to_csv(DOCS_DIR / "data_quality_report.csv", index=False)
display(quality_report)

# Proof that Stage 3 changed nothing
assert raw.shape == RAW_SHAPE_BEFORE_EDA
assert int(pd.util.hash_pandas_object(raw, index=True).sum()) == RAW_HASH_BEFORE_EDA
assert md5(RAW_FILE) == RAW_MD5_START
print("Stage 3 changed nothing: raw data in memory and raw file on disk are identical to the start.")
```

Output

	Problem	Columns affected	Records affected	Severity	Proposed treatment
0	Target is empty	waiting_time_min	168	High	Remove the records (a record without a target ...
1	Leakage: known only after the outcome	consultation_duration_min, attended	1200	High	Drop both columns
2	Empty cells in feature columns	facility_type, visit_date, patient_age_years, ...	193	Medium	Impute after the split with values learned fro...
3	Disguised value: the word 'None'	insurance_type	137	Medium	Keep as the category 'None' = no insurance (as...
4	Full duplicate rows / repeated visit_id	all / visit_id	5	Medium	Keep the first copy, remove the rest
5	Same values under a different visit_id	all	0	-	Checked: none found
6	Mixed upper/lower case	sex, referral, attended	212	Medium	Write in one form (Title case)
7	Extra spaces at the end of text	district, facility_code, insurance_type	30	Medium	Remove the spaces
8	Wrong data type	visit_date (text)	1200	Low	Convert to a date
9	Inconsistent date formats or units	visit_date, numerical columns	0	-	Checked: none found
10	day_of_week different from visit_date	day_of_week, visit_date	0	-	Checked: none found
11	Values outside the allowed range	patient_age_years, arrival_hour, distance_to_f...	0	-	Checked: none found
12	Sentinel numbers or impossible zeros	numerical columns	0	-	Checked: none found
13	Illogical: Child Health patient aged 18+	service_department, patient_age_years	105	Medium	Keep and report as a limitation (cannot tell w...
14	Illogical: male patient in Maternal Health	service_department, sex	77	Medium	Keep and report as a limitation (cannot tell w...
15	Facility code used in several districts	facility_code	8	Low	Drop the column (identifier-like)
16	Outliers by the IQR rule (all are possible val...	distance, travel time, previous visits, consul...	88	Low	Keep; reduce the tails of skewed columns by tr...
17	Source stated as NISR	whole file	1200	High	Record in the Source

	but not verified; link, ...				Log; the group must add t...

```
Stage 3 changed nothing: raw data in memory and raw file on disk are identical to the start.
```

What this does. Gathers every finding of Stage 3 into the Data Quality Report, with the counts taken from the checks above, and saves it to `docs/data_quality_report.csv`. The last lines prove that the audit changed neither the data in memory nor the file on disk.

What we found. The table lists only what the data shows. Checks that found nothing are kept in the table and marked "Checked: none found". The most serious problems are the 168 records without a target, the two leakage columns and the missing documentation of the source.

What we decided. The "Proposed treatment" column is the plan for Stage 4.

Stage 4 - Clean and pre-process (PC 1.3)

Each step is one small function. After each step we show a before/after view and add one line to the Cleaning Log.

Code - cell 23

```
cleaning_log = []


def log_step(operation, columns, action, reason, records_affected, before, after):
    """Add one line to the Cleaning Log."""
    cleaning_log.append({"Step": len(cleaning_log) + 1, "Operation": operation, "Columns": columns, "Action": action,
                         "Reason": reason, "Records affected": int(records_affected), "Before": before, "After":
after})


def before_after(before, after, cols=()):
    """Print shape, empty cells and (for short text columns) the different values, before and after a step."""
    print(f"Shape         before {before.shape}  ->  after {after.shape}")
    print(f"Empty cells   before {int(before.isna().sum().sum())}  ->  after {int(after.isna().sum().sum())}")
    for c in cols:
        b = before[c] if c in before.columns else None
        a = after[c] if c in after.columns else None
        for label, s in (("before", b), ("after ", a)):
            if s is None:
                print(f"  {c} {label}: column not present")
            elif s.dtype == object and s.nunique() <= 12:
                print(f"  {c} {label}: {s.isna().sum()} empty, values {sorted(s.dropna().unique().tolist())}")
            else:
                print(f"  {c} {label}: {s.isna().sum()} empty, {s.nunique()} different values, type {s.dtype}")


df = raw.copy()      # all cleaning happens on this copy; raw stays as loaded
print("Working copy:", df.shape)
```

Output

```
Working copy: (1200, 20)
```

What this does. Creates the empty Cleaning Log, a helper that adds a line to it, and a helper that prints the before/after view. It then makes the working copy `df`. The variable `raw` is not touched again, except to read from it.

4.1 Tidy the text labels

Code - cell 24

```
CASE_COLS = ["sex", "referral", "attended"]


def tidy_text(data):
    """Remove spaces around text values and write sex, referral and attended in one form (Male, Yes)."""
    data = data.copy()
    for c in data.select_dtypes("object").columns:
        data[c] = data[c].str.strip()
    for c in CASE_COLS:
        if c in data.columns and data[c].dtype == object:
            data[c] = data[c].str.title()
    return data


before = df
df = tidy_text(df)
changed = int(((before != df) & before.notna()).any(axis=1).sum())
before_after(before, df, ["sex", "referral", "attended", "insurance_type", "district"])
log_step("Tidy text labels", "all text columns; sex, referral, attended",
         "Removed spaces around values; wrote sex, referral and attended in Title case",
         "The same value was written in several ways", changed,
         f"district {before['district'].nunique()} values, sex {before['sex'].nunique()}, insurance_type
{before['insurance_type'].nunique()}",
         f"district {df['district'].nunique()} values, sex {df['sex'].nunique()}, insurance_type
{df['insurance_type'].nunique()}")
print("Records changed:", changed)
```

Output

```
Shape         before (1200, 20)  ->  after (1200, 20)
Empty cells   before 570  ->  after 570
  sex before: 0 empty, values ['FEMALE', 'Female', 'MALE', 'Male', 'female', 'male']
  sex after : 0 empty, values ['Female', 'Male']
  referral before: 12 empty, values ['NO', 'No', 'YES', 'Yes', 'no', 'yes']
  referral after : 12 empty, values ['No', 'Yes']
  attended before: 14 empty, values ['NO', 'No', 'YES', 'Yes', 'no', 'yes']
  attended after : 14 empty, values ['No', 'Yes']
  insurance_type before: 39 empty, values ['Mutuelle de Santé', 'Mutuelle de Santé ', 'None', 'None ', 'Private',
'RSSB', 'RSSB ']
```

```
  insurance_type after : 39 empty, values ['Mutuelle de Santé', 'None', 'Private', 'RSSB']
  district before: 0 empty, 29 different values, type object
  district after : 0 empty, 22 different values, type object
Records changed: 227
```

What this does. `tidy_text()` removes spaces at the start and end of every text value and writes `sex`, `referral` and `attended` with a capital first letter.

What we found. 227 records had at least one value changed. `sex`, `referral` and `attended` now have two values each, `district` has 22 values instead of 29, and `insurance_type` has four values (`Mutuelle de Santé`, `None`, `Private`, `RSSB`). The number of empty cells did not change.

What we decided. None stays as a category of `insurance_type` (no insurance), following the assumption in section 3.1.

4.2 Remove duplicates

Code - cell 25

```
def remove_duplicates(data):
    """Remove rows that are exact copies of an earlier row."""
    return data.drop_duplicates(keep="first")


before = df
df = remove_duplicates(df)
removed = len(before) - len(df)
before_after(before, df)
print("Repeated visit_id after the step:", int(df["visit_id"].duplicated().sum()))
assert df["visit_id"].is_unique
log_step("Remove duplicates", "all", "Kept the first copy of each fully duplicated row and removed the others",
         "A visit must be counted once", removed, f"{len(before)} rows, {int(before['visit_id'].duplicated().sum())}
repeated visit_id",
         f"{len(df)} rows, {int(df['visit_id'].duplicated().sum())} repeated visit_id")
```

Output

```
Shape         before (1200, 20)  ->  after (1195, 20)
Empty cells   before 570  ->  after 567
Repeated visit_id after the step: 0
```

What this does. `remove_duplicates()` keeps the first copy of each fully duplicated row. It runs after the text is tidied, so that a copy that differs only by a space or a capital letter would also be caught.

What we found. 5 rows were removed (1,200 to 1,195). Every `visit_id` is now unique. No merging was needed because the repeated codes were exact copies.

4.3 Correct types, formats and units

Code - cell 26

```
def fix_types(data):
    """Convert visit_date from text to a real date. Anything that is not a valid date becomes empty."""
    data = data.copy()
    data["visit_date"] = pd.to_datetime(data["visit_date"], format="%Y-%m-%d", errors="coerce")
    return data


before = df
df = fix_types(df)
new_empty = int(df["visit_date"].isna().sum() - before["visit_date"].isna().sum())
before_after(before, df, ["visit_date"])
print("Dates that could not be converted:", new_empty)
log_step("Correct types", "visit_date", "Converted text to a date (YYYY-MM-DD)", "Needed to take the month from the
date",
         df["visit_date"].notna().sum(), f"type {before['visit_date'].dtype}, {int(before['visit_date'].isna().sum())}
empty",
         f"type {df['visit_date'].dtype}, {int(df['visit_date'].isna().sum())} empty")
log_step("Correct formats and units", "all numerical columns", "No change",
         "Checked in Stage 3: one date format, no mixed units, ages are whole numbers", 0, "none found", "no change")
```

Output

```
Shape         before (1195, 20)  ->  after (1195, 20)
Empty cells   before 567  ->  after 567
  visit_date before: 8 empty, 579 different values, type object
  visit_date after : 8 empty, 579 different values, type datetime64[ns]
Dates that could not be converted: 0
```

What this does. `fix_types()` turns `visit_date` from text into a real date.

What we found. All 1,187 dates converted; the 8 empty dates stay empty and no new empty value was created. Formats and units needed no correction (Stage 3 found one date format and no mixed units), and this is written in the log with 0 records affected.

What we decided. `patient_age_years` and `previous_visits_12m` hold whole numbers but stay as decimal type for now, because a whole-number column cannot hold empty cells in standard pandas. They have no empty cells after imputation.

4.4 Rebuild facility_type from the facility code

Code - cell 27

```
PREFIX_TO_TYPE = {"HC": "Health Centre", "DH": "District Hospital", "RH": "Referral Hospital"}


def repair_facility_type(data):
    """Fill an empty facility_type from the first two letters of facility_code (HC, DH, RH)."""
    data = data.copy()
    from_code = data["facility_code"].str[:2].map(PREFIX_TO_TYPE)
    data["facility_type"] = data["facility_type"].fillna(from_code)
    return data


before = df
df = repair_facility_type(df)
filled = int(before["facility_type"].isna().sum() - df["facility_type"].isna().sum())
before_after(before, df, ["facility_type"])
print(df["facility_type"].value_counts().to_string())
log_step("Rebuild facility_type", "facility_type (from facility_code)", "Filled empty facility_type from the code
prefix HC/DH/RH",
         "The prefix agrees with facility_type in every record where both are present (Stage 3), so this is a lookup
and not a guess",
         filled, f"{int(before['facility_type'].isna().sum())} empty", f"{int(df['facility_type'].isna().sum())}
empty")
```

Output

```
Shape         before (1195, 20)  ->  after (1195, 20)
Empty cells   before 567  ->  after 557
  facility_type before: 10 empty, values ['District Hospital', 'Health Centre', 'Referral Hospital']
  facility_type after : 0 empty, values ['District Hospital', 'Health Centre', 'Referral Hospital']
facility_type
Health Centre        824
District Hospital    335
Referral Hospital     36
```

What this does. `repair_facility_type()` fills an empty `facility_type` using the first two letters of `facility_code`: HC = Health Centre, DH = District Hospital, RH = Referral Hospital.

What we found. All 10 empty values were filled; `facility_type` now has no gaps.

What we decided. This is done before the split because it is a fixed rule, not a statistic learned from the data: Stage 3 showed that the prefix and the type agree in every record where both exist. It is also needed because we stratify the split by `facility_type`. The meaning of the prefixes is an assumption.

4.5 Set impossible values to missing

Code - cell 28

```
def impossible_to_missing(data):
    """Replace any value outside the allowed range (VALID_RANGES) with an empty cell."""
    data = data.copy()
    for col, (low, high) in VALID_RANGES.items():
        if col in data.columns:
            bad = (data[col] < low) | (data[col] > high)
            data.loc[bad, col] = np.nan
    return data


before = df
df = impossible_to_missing(df)
set_missing = int(df.isna().sum().sum() - before.isna().sum().sum())
before_after(before, df)
print("Values set to missing:", set_missing)
log_step("Impossible values to missing", ", ".join(VALID_RANGES), "Values outside the allowed range are set to empty",
         "An impossible value is an error and must not be used", set_missing, "none found", "no change")
log_step("Illogical department records", "service_department, patient_age_years, sex", "No change (kept)",
         "Adults in Child Health and men in Maternal Health: we cannot tell which column is wrong, so changing them
```

```
would invent data",
        0, "reported in Stage 3", "kept; listed as a limitation")
```

Output

```
Shape         before (1195, 20)  ->  after (1195, 20)
Empty cells   before 557  ->  after 557
Values set to missing: 0
```

What this does. `impossible_to_missing()` replaces any value outside the allowed range of the Data Dictionary with an empty cell, to be imputed later.

What we found. 0 values were outside the ranges, so nothing changed (checked: none found). The function is kept because new records in the future may contain impossible values.

What we decided. The illogical department records (section 3.4) are kept unchanged and logged with 0 records affected.

4.6 Decide what to do with outliers

Code - cell 29

```
decisions = {
    "waiting_time_min": "Keep. It is the target; long waits are exactly what the manager wants to predict.",
    "distance_to_facility_km": "Keep. All values are possible (highest 25.3 km). Reduce the tail by transformation in
Stage 5.",
    "estimated_travel_time_min": "Keep. All values are possible (highest 117 min). Reduce the tail by transformation in
Stage 5.",
    "previous_visits_12m": "Keep. Up to 8 visits in a year is possible for a patient with a long-term illness.",
    "patient_age_years": "Keep. A patient aged 90 is possible.",
    "arrival_hour": "Keep. Arrivals at 16:00 are possible.",
    "consultation_duration_min": "Not relevant. The column is dropped as leakage in the next step.",
}
outlier_decisions = outlier_table[["Column", "Upper limit", "Above upper limit", "Highest value"]].copy()
outlier_decisions["Decision"] = outlier_decisions["Column"].map(decisions)
display(outlier_decisions)
log_step("Outlier decision", "all numerical columns", "Kept all values (no removal, no capping)",
         "All flagged values are possible; skewed columns are transformed in Stage 5", 0,
         f"{int(outlier_table['Above upper limit'].sum())} values above the IQR limits", "unchanged")
```

Output

	Column	Upper limit	Above upper limit	Highest value	Decision
0	patient_age_years	86.00	1	90.0	Keep. A patient aged 90 is possible.
1	arrival_hour	15.50	2	16.0	Keep. Arrivals at 16:00 are possible.
2	distance_to_facility_km	15.55	37	25.3	Keep. All values are possible (highest 25.3 km...
3	estimated_travel_time_min	75.50	27	117.0	Keep. All values are possible (highest 117 min...
4	previous_visits_12m	6.00	12	8.0	Keep. Up to 8 visits in a year is possible for...
5	waiting_time_min	96.00	4	105.0	Keep. It is the target; long waits are exactly...
6	consultation_duration_min	33.50	5	41.0	Not relevant. The column is dropped as leakage...

What this does. Records the decision for each numerical column using the IQR table from Stage 3. No data is changed in this cell.

What we decided. Keep every value. Reason: the IQR rule flags values that are unusual, not values that are wrong, and Stage 3 showed that all flagged values are possible. Removing them would hide the patients who live far away, who are part of BQ3. The long tails of distance and travel time are handled in Stage 5 by a transformation chosen on the training set.

4.7 Drop leakage and identifier columns

Code - cell 30

```
LEAKAGE_COLS = ["consultation_duration_min", "attended"]
IDENTIFIER_COLS = ["facility_code"]          # visit_id becomes the row label and is not a feature
```

```
def drop_leakage_and_identifiers(data):
    """Drop columns known only after the outcome and identifier columns. visit_id is kept only as the row label."""
    data = data.drop(columns=[c for c in LEAKAGE_COLS + IDENTIFIER_COLS if c in data.columns])
    return data.set_index("visit_id")


before = df
df = drop_leakage_and_identifiers(df)
before_after(before, df, ["consultation_duration_min", "attended", "facility_code"])
print("Columns now:", df.columns.tolist())
log_step("Drop leakage columns", ", ".join(LEAKAGE_COLS), "Dropped", "Known only after the wait has ended (Stage 3.8)",
         len(df), f"{before.shape[1]} columns", f"{before.shape[1] - len(LEAKAGE_COLS)} columns")
log_step("Drop identifier columns", "visit_id, facility_code", "facility_code dropped; visit_id moved to the row
label",
         "Identifiers describe the record, not the visit, and must not be used as features", len(df),
         f"{before.shape[1] - len(LEAKAGE_COLS)} columns", f"{df.shape[1]} columns + row label")
```

Output

```
Shape         before (1195, 20)  ->  after (1195, 16)
Empty cells   before 557  ->  after 370
  consultation_duration_min before: 174 empty, 33 different values, type float64
  consultation_duration_min after : column not present
  attended before: 13 empty, values ['No', 'Yes']
  attended after : column not present
  facility_code before: 0 empty, 138 different values, type object
  facility_code after : column not present
Columns now: ['province', 'district', 'facility_type', 'visit_date', 'patient_age_years', 'sex', 'insurance_type',
'service_department', 'appointment_type', 'arrival_hour', 'day_of_week', 'distance_to_facility_km',
'estimated_travel_time_min', 'referral', 'previous_visits_12m', 'waiting_time_min']
```

What this does. `drop_leakage_and_identifiers()` removes `consultation_duration_min` and `attended` (leakage) and `facility_code` (identifier-like code). `visit_id` becomes the row label so that we can follow each record through the split; it is not a feature and is not written to the final files.

What we found. 16 columns remain: 15 possible features and the target.

What we decided. `facility_code` is dropped because the same code is used in several districts and because each facility has only a handful of records (too few to learn anything about a single facility). The useful part of the code, the level of the facility, is already in `facility_type`.

4.8 Remove records with no target

Code - cell 31

```
def remove_no_target(data):
    """Remove records that have no waiting time."""
    return data.dropna(subset=[TARGET])


before = df
df = remove_no_target(df)
removed = len(before) - len(df)
before_after(before, df, [TARGET])
log_step("Remove records with no target", TARGET, "Removed records with an empty waiting time",
         "A record without a target cannot be used to train or test a model", removed,
         f"{len(before)} rows, {int(before[TARGET].isna().sum())} without target", f"{len(df)} rows, 0 without target")
print("Records removed:", removed, "| Records left:", len(df))
```

Output

```
Shape         before (1195, 16)  ->  after (1027, 16)
Empty cells   before 370  ->  after 175
  waiting_time_min before: 168 empty, 94 different values, type float64
  waiting_time_min after : 0 empty, 94 different values, type float64
Records removed: 168 | Records left: 1027
```

What this does. `remove_no_target()` removes the records with no waiting time.

What we found. 168 records were removed, leaving 1,027 records. These are mostly visits where the patient did not attend, so no wait was measured.

What we decided. We do not impute a target: an invented waiting time would teach a model a value that never happened. 1,027 usable records is just above the 1,000 minimum, and this is noted as a risk in the Data Card.

4.9 Save the clean file and show the Cleaning Log

Code - cell 32

```
clean = df.copy()
CLEAN_FILE = INTERIM_DIR / "visits_clean.csv"
clean.to_csv(CLEAN_FILE, encoding="utf-8")

cleaning_log_table = pd.DataFrame(cleaning_log)
cleaning_log_table.to_csv(DOCS_DIR / "cleaning_log.csv", index=False)
display(cleaning_log_table)

print("Clean file:", CLEAN_FILE.relative_to(ROOT), clean.shape)
print("\nEmpty cells left for imputation after the split:")
print(clean.isna().sum()[clean.isna().sum() > 0].to_string())
```

Output

	Step	Operation	Columns	Action	Reason	Records affected	Before	After
0	1	Tidy text labels	all text columns; sex, referral, attended	Removed spaces around values; wrote sex, refer...	The same value was written in several ways	227	district 29 values, sex 6, insurance_type 7	district 22 values, sex 2, insurance_type 4
1	2	Remove duplicates	all	Kept the first copy of each fully duplicated r...	A visit must be counted once	5	1200 rows, 5 repeated visit_id	1195 rows, 0 repeated visit_id
2	3	Correct types	visit_date	Converted text to a date (YYYY-MM-DD)	Needed to take the month from the date	1187	type object, 8 empty	type datetime64[ns], 8 empty
3	4	Correct formats and units	all numerical columns	No change	Checked in Stage 3: one date format, no mixed ...	0	none found	no change
4	5	Rebuild facility_type	facility_type (from facility_code)	Filled empty facility_type from the code prefi...	The prefix agrees with facility_type in every ...	10	10 empty	0 empty
5	6	Impossible values to missing	patient_age_years, arrival_hour, distance_to_f...	Values outside the allowed range are set to empty	An impossible value is an error and must not b...	0	none found	no change
6	7	Illogical department records	service_department, patient_age_years, sex	No change (kept)	Adults in Child Health and men in Maternal Hea...	0	reported in Stage 3	kept; listed as a limitation
7	8	Outlier decision	all numerical columns	Kept all values (no removal, no capping)	All flagged values are possible; skewed column...	0	88 values above the IQR limits	unchanged
8	9	Drop leakage columns	consultation_duration_min, attended	Dropped	Known only after the wait has ended (Stage 3.8)	1195	20 columns	18 columns
9	10	Drop identifier columns	visit_id, facility_code	facility_code dropped; visit_id moved to the r...	Identifiers describe the record, not the visit...	1195	18 columns	16 columns + row label
10	11	Remove records with no target	waiting_time_min	Removed records with an empty waiting time	A record without a target cannot be used to tr...	168	1195 rows, 168 without target	1027 rows, 0 without target

```
Clean file: data\interim\visits_clean.csv (1027, 16)

Empty cells left for imputation after the split:
visit_date                   5
patient_age_years           19
insurance_type              34
service_department          21
appointment_type            14
```

```
distance_to_facility_km      31
estimated_travel_time_min    27
referral                      9
previous_visits_12m          15
```

What this does. Saves the cleaned data to `data/interim/visits_clean.csv` and the Cleaning Log to `docs/cleaning_log.csv`.

What we found. The clean file has 1,027 rows and 16 columns (plus the `visit_id` row label). Nine feature columns still have empty cells. These are left empty on purpose.

What we decided. Empty feature values are filled only after the split, using values learned from the training set, so that nothing from the validation or test records influences the fill values.

4.10 Split 70 / 15 / 15 before imputing or scaling

Code - cell 33

```
STRATIFY_COL = "facility_type"

train_raw, temp = train_test_split(clean, test_size=0.30, random_state=SEED, stratify=clean[STRATIFY_COL])
val_raw, test_raw = train_test_split(temp, test_size=0.50, random_state=SEED, stratify=temp[STRATIFY_COL])

# No record may be in two sets, and no record may be lost
assert len(set(train_raw.index) & set(val_raw.index)) == 0
assert len(set(train_raw.index) & set(test_raw.index)) == 0
assert len(set(val_raw.index) & set(test_raw.index)) == 0
assert len(train_raw) + len(val_raw) + len(test_raw) == len(clean)
assert set(train_raw.index) | set(val_raw.index) | set(test_raw.index) == set(clean.index)

split_table = pd.DataFrame({
    "Records": [len(train_raw), len(val_raw), len(test_raw)],
    "Share %": [round(100 * len(s) / len(clean), 1) for s in (train_raw, val_raw, test_raw)],
}, index=["train", "validation", "test"])
for level in ["Health Centre", "District Hospital", "Referral Hospital"]:
    split_table[f"{level} %"] = [round(100 * (s[STRATIFY_COL] == level).mean(), 1) for s in (train_raw, val_raw,
test_raw)]
display(split_table)
print("No record is in two sets: confirmed.")
```

Output

	Records	Share %	Health Centre %	District Hospital %	Referral Hospital %
train	718	69.9	69.5	27.6	2.9
validation	154	15.0	69.5	27.3	3.2
test	155	15.1	69.7	27.7	2.6

```
No record is in two sets: confirmed.
```

What this does. Splits the clean data into training (70%), validation (15%) and test (15%) with random seed 42. It is done in two steps: first 70/30, then the 30% is cut in half. The `assert` lines stop the notebook if a record is in two sets or is lost.

What we found. 718 training, 154 validation and 155 test records (69.9%, 15.0%, 15.1%). The share of each facility type is almost the same in the three sets (about 69% health centres, 28% district hospitals, 3% referral hospitals).

What we decided. The target is a number, so we stratify by a key category. We chose `facility_type` because it is the column most strongly linked to waiting time in the audit and because referral hospitals are rare (about 3%); without stratifying, one set could receive almost none of them. **From here on the test set is not used for any decision.**

4.11 Impute with values learned from the training set only

Code - cell 34

```
NUM_BASE = ["patient_age_years", "arrival_hour", "distance_to_facility_km", "estimated_travel_time_min",
"previous_visits_12m"]
CAT_BASE = ["province", "district", "facility_type", "sex", "insurance_type", "service_department",
            "appointment_type", "day_of_week", "referral"]


def learn_imputation(train):
    """Learn the fill values from the training set only."""
    p = {"median": {c: float(train[c].median()) for c in NUM_BASE},
         "mode": {c: train[c].mode()[0] for c in CAT_BASE}}
    # Group median for travel time: the median travel time inside each distance band (bands = training quartiles)
```

```
    q = train["distance_to_facility_km"].quantile([0.25, 0.50, 0.75]).tolist()
    p["distance_band_edges"] = [-np.inf] + q + [np.inf]
    bands = pd.cut(train["distance_to_facility_km"], p["distance_band_edges"], labels=False)
    p["travel_median_by_band"] = train.groupby(bands)["estimated_travel_time_min"].median().to_dict()
    return p


def apply_imputation(data, p):
    """Fill empty cells with the values learned by learn_imputation()."""
    data = data.copy()
    for c in ["patient_age_years", "arrival_hour", "distance_to_facility_km", "previous_visits_12m"]:
        data[c] = data[c].fillna(p["median"][c])
    bands = pd.cut(data["distance_to_facility_km"], p["distance_band_edges"], labels=False)
    data["estimated_travel_time_min"] = (data["estimated_travel_time_min"]
                                         .fillna(bands.map(p["travel_median_by_band"]))
                                         .fillna(p["median"]["estimated_travel_time_min"]))
    for c in CAT_BASE:
        data[c] = data[c].fillna(p["mode"][c])
    return data


impute_params = learn_imputation(train_raw)
train_imp, val_imp, test_imp = (apply_imputation(s, impute_params) for s in (train_raw, val_raw, test_raw))

imputation_table = pd.DataFrame({
    "Empty in train": train_raw.isna().sum(), "Empty in validation": val_raw.isna().sum(), "Empty in test":
test_raw.isna().sum(),
}).drop(index=[TARGET])
imputation_table["Method"] = ["mode", "mode", "mode (already filled from code)", "not imputed here (see 5.1)",
"median", "mode", "mode",
                              "mode", "mode", "median", "mode", "median", "group median by distance band", "mode",
"median"]
imputation_table["Value learned from train"] = [
    impute_params["mode"].get(c, impute_params["median"].get(c, "-")) if c != "estimated_travel_time_min"
    else "per band: " + str({int(k): v for k, v in impute_params["travel_median_by_band"].items()})
    for c in imputation_table.index]
display(imputation_table)

feature_cols_now = [c for c in train_imp.columns if c not in [TARGET, "visit_date"]]
for name, s in (("train", train_imp), ("validation", val_imp), ("test", test_imp)):
    assert s[feature_cols_now].isna().sum().sum() == 0
    print(f"{name}: empty cells left (excluding visit_date) = {int(s[feature_cols_now].isna().sum().sum())}, "
          f"visit_date empty = {int(s['visit_date'].isna().sum())}")
```

Output

	Empty in train	Empty in validation	Empty in test	Method	Value learned from train
province	0	0	0	mode	Southern
district	0	0	0	mode	Huye
facility_type	0	0	0	mode (already filled from code)	Health Centre
visit_date	4	1	0	not imputed here (see 5.1)	-
patient_age_years	12	4	3	median	30.0
sex	0	0	0	mode	Male
insurance_type	23	4	7	mode	Mutuelle de Santé
service_department	16	3	2	mode	General Outpatient
appointment_type	9	0	5	mode	Walk-in
arrival_hour	0	0	0	median	9.0
day_of_week	0	0	0	mode	Sunday
distance_to_facility_km	25	5	1	median	5.0
estimated_travel_time_min	18	2	7	group median by distance band	per band: {0: 11.0, 1: 18.0, 2: 29.0, 3: 46.0}
referral	6	3	0	mode	No
previous_visits_12m	8	4	3	median	2.0

```
train: empty cells left (excluding visit_date) = 0, visit_date empty = 4
validation: empty cells left (excluding visit_date) = 0, visit_date empty = 1
test: empty cells left (excluding visit_date) = 0, visit_date empty = 0
```

What this does. `learn_imputation()` reads the training set only and stores one fill value per column. `apply_imputation()` uses those stored values to fill the training, validation and test sets in the same way. The table shows how many cells were empty in each set, the method and the value learned.

Justification per column.

Column(s)	Method	Why
`patient_age_years`, `previous_visits_12m`	Median of the training set	Few gaps (under 2%). The median is a real,

		typical value and is not pulled by unusual values.
distance_to_facility_km	Median of the training set	The column has a long right tail, so the mean would be too high.
estimated_travel_time_min	Group median: median travel time of training records in the same distance band (four bands cut at the training quartiles of distance)	Travel time follows distance closely (correlation 0.88 in the audit), so a patient 12 km away should not receive the travel time of a typical patient 5 km away.
insurance_type, service_department, appointment_type, referral	Mode (most frequent value) of the training set	They are categories, so a median does not exist. Gaps are small (under 4%).
Columns with no gaps today (province, sex, arrival_hour, ...)	A fill value is still learned	So that prepare_new_data() does not fail if a future record has a gap.
visit_date	Not filled	A date cannot be guessed. Only the month is taken from it in Stage 5, and a missing month is filled there with the training mode.

What we found. After imputation no feature cell is empty in any of the three sets, apart from the dates that are still missing (handled in section 5.1).

What we decided. The fill values come from the 718 training records only. The validation and test sets are filled with the training values and never contribute to them.

4.12 Encode the categories

Code - cell 35

```
FACILITY_ORDER = ["Health Centre", "District Hospital", "Referral Hospital"]     # ordered: level of care
NOMINAL = ["province", "sex", "insurance_type", "service_department", "appointment_type", "day_of_week", "referral"]


def clean_name(text):
    """Make a safe column name: no accents, no spaces, lower case."""
    text = unicodedata.normalize("NFKD", text).encode("ascii", "ignore").decode()
    return re.sub(r"[^0-9a-zA-Z]+", "_", text).strip("_").lower()


def fit_encoders(train):
    """Learn the categories from the training set only."""
    ordinal = OrdinalEncoder(categories=[FACILITY_ORDER]).fit(train[["facility_type"]])
    onehot = OneHotEncoder(drop="first", sparse_output=False, handle_unknown="ignore").fit(train[NOMINAL])
    return ordinal, onehot


def apply_encoders(data, ordinal, onehot):
    """Return the numerical base columns, the ordinal facility level and the one-hot columns."""
    out = data[NUM_BASE].copy()
    out["facility_level"] = ordinal.transform(data[["facility_type"]])[:, 0].astype(int)
    names = [clean_name(n) for n in onehot.get_feature_names_out(NOMINAL)]
    dummies = pd.DataFrame(onehot.transform(data[NOMINAL]), columns=names, index=data.index).astype(int)
    return pd.concat([out, dummies], axis=1)


ordinal_encoder, onehot_encoder = fit_encoders(train_imp)
train_enc, val_enc, test_enc = (apply_encoders(s, ordinal_encoder, onehot_encoder) for s in (train_imp, val_imp,
test_imp))

print("Shape before encoding:", train_imp.drop(columns=[TARGET]).shape, "-> after:", train_enc.shape)
print("\nReference category of each one-hot group (the category that gets no column):")
for col, cats in zip(NOMINAL, onehot_encoder.categories_):
    print(f"  {col}: reference = {cats[0]} | columns for {list(cats[1:])}")
display(train_enc.head(3))
```

Output

```
Shape before encoding: (718, 15) -> after: (718, 30)

Reference category of each one-hot group (the category that gets no column):
  province: reference = Eastern | columns for ['Kigali City', 'Northern', 'Southern', 'Western']
  sex: reference = Female | columns for ['Male']
  insurance_type: reference = Mutuelle de Santé | columns for ['None', 'Private', 'RSSB']
  service_department: reference = Child Health | columns for ['Dental', 'Emergency', 'Eye Clinic', 'General
Outpatient', 'Maternal Health', 'NCD Clinic']
  appointment_type: reference = Follow-up | columns for ['Referral', 'Scheduled', 'Walk-in']
  day_of_week: reference = Friday | columns for ['Monday', 'Saturday', 'Sunday', 'Thursday', 'Tuesday', 'Wednesday']
  referral: reference = No | columns for ['Yes']
```

```
             patient_age_years  arrival_hour  distance_to_facility_km  estimated_travel_time_min  previous_visits_12m  facility_level
province_kigali_city  province_northern  province_southern  \
visit_id
```

```
RW-HLTH-00959               13.0          12.0                    16.5                      84.0                 4.0               0
0                  1                  0
RW-HLTH-00381               40.0          10.0                     7.2                      27.0                 4.0               0
0                  0                  1
RW-HLTH-00963               38.0           7.0                     3.2                      25.0                 2.0               1
0                  0                  1

               province_western  sex_male  insurance_type_none  insurance_type_private  insurance_type_rssb  service_department_dental
service_department_emergency  service_department_eye_clinic  \
visit_id
RW-HLTH-00959                 0         0                    0                       0                    1                         0
0                              0
RW-HLTH-00381                 0         1                    0                       0                    0                         0
0                              0
RW-HLTH-00963                 0         1                    0                       0                    0                         0
0                              0

               service_department_general_outpatient  service_department_maternal_health  service_department_ncd_clinic
appointment_type_referral  appointment_type_scheduled  \
visit_id
RW-HLTH-00959                                      0                                  1                             0
0                          1
RW-HLTH-00381                                      1                                  0                             0
0                          0
RW-HLTH-00963                                      1                                  0                             0
0                          1

               appointment_type_walk_in  day_of_week_monday  day_of_week_saturday  day_of_week_sunday  day_of_week_thursday
day_of_week_tuesday  day_of_week_wednesday  referral_yes
visit_id
RW-HLTH-00959                         0                   0                     0                   0                     0
0                    1                  0
RW-HLTH-00381                         1                   1                     0                   0                     0
0                    0                  1
RW-HLTH-00963                         0                   0                     0                   0                     0
1                    0                  0
```

What this does. Turns text categories into numbers, with encoders fitted on the training set only.

- **Ordinal** for `facility_type`, because its values have a natural order (level of care): Health Centre = 0, District Hospital = 1, Referral Hospital = 2. The new column is called `facility_level`.
- **One-hot** for the unordered categories (`province`, `sex`, `insurance_type`, `service_department`, `appointment_type`, `day_of_week`, `referral`): one 0/1 column per category.

What we found. The 15 feature columns become 30 numerical columns: 5 numerical base columns, `facility_level` and 24 one-hot columns.

What we decided.

- One category per group is left out as the reference (`drop="first"`), because the last column of a group can always be worked out from the others. The printed list shows the reference of each group (for example Eastern for province, Female for sex).
- Column names are made safe (lower case, no spaces or accents), for example `province_kigali_city` and `appointment_type_walk_in`.
- `district` (22 values, about 33 training records each) is **not** one-hot encoded: it would add 21 thin columns. Its information is carried by `province` and by a district feature built in Stage 5. `visit_date` is used in Stage 5 to take the month.
- **Scaling** is the last numerical step. It is done in section 5.3, after the new features and the skewness transformations, because a ratio of scaled values or the logarithm of a scaled value has no meaning. The scaler is still fitted on the training set only.

4.13 Is the target imbalanced?

Code - cell 36

```
balance = pd.DataFrame({
    "Records": [len(s) for s in (train_imp, val_imp, test_imp)],
    "Mean target": [s[TARGET].mean() for s in (train_imp, val_imp, test_imp)],
    "Median target": [s[TARGET].median() for s in (train_imp, val_imp, test_imp)],
    "Std target": [s[TARGET].std() for s in (train_imp, val_imp, test_imp)],
}, index=["train", "validation", "test"]).round(2)
display(balance)
print("Training skewness of the target:", round(train_imp[TARGET].skew(), 2))
level_share = train_imp["facility_type"].value_counts()
print("\nTraining records per facility type:")
print(level_share.to_string())
```

```
print("Ratio health centres : referral hospitals =", round(level_share["Health Centre"] / level_share["Referral
Hospital"], 1), ": 1")
```

Output

	Records	Mean target	Median target	Std target
train	718	40.57	40.0	19.48
validation	154	41.11	41.0	19.23
test	155	40.19	41.0	20.87

```
Training skewness of the target: 0.29

Training records per facility type:
facility_type
Health Centre        499
District Hospital    198
Referral Hospital     21
Ratio health centres : referral hospitals = 23.8 : 1
```

What this does. Checks whether the target or a key group is unbalanced. The table of the three sets is descriptive only; no decision is taken from the test row.

What we found. The target is a continuous number, so there are no classes and no class ratio. Its training skewness is small (0.29), so no range of waiting times is badly under-represented. The mean and spread of the target are similar in the three sets. The imbalance that does exist is between groups: in the training set there are about 24 health-centre records for every referral-hospital record (499 against 21).

What we decided. No resampling. The planned remedy for the group imbalance is (1) the stratified split, already done, which keeps the same share of each facility type in every set, and (2) a recommendation to the analyst who trains the model to report the error separately per facility type and, if needed, to use sample weights on the training set only. The validation and test sets are never resampled.

Stage 5 - Features (PC 1.4)

All choices in this stage (peak hours, group averages, transformations, selection) are learned from the **training set only** and then applied unchanged to validation and test.

5.1 Create new features

Code - cell 37

```
MIN_VISITS_PER_HOUR = 30      # assumption: an hour needs at least 30 training visits before we trust its average
AGE_BINS = [-0.1, 4, 17, 39, 59, 200]      # 0-4, 5-17, 18-39, 40-59, 60+
MIN_GROUP_SIZE = 5            # assumption: a facility-type x department group needs at least 5 training visits

by_hour = train_imp.groupby("arrival_hour")[TARGET].agg(["count", "mean"]).round(1)
by_hour["peak hour?"] = (by_hour["count"] >= MIN_VISITS_PER_HOUR) & (by_hour["mean"] > train_imp[TARGET].mean())
print("Training mean waiting time:", round(train_imp[TARGET].mean(), 1))
display(by_hour.T)


def learn_feature_params(train):
    """Learn from the training set what the features need: peak hours, group medians and the usual month."""
    overall_mean = train[TARGET].mean()
    hours = train.groupby("arrival_hour")[TARGET].agg(["count", "mean"])
    level_dept = train.groupby(train["facility_type"] + " | " + train["service_department"])[TARGET].agg(["count",
"median"])
    return {
        "peak_hours": sorted(int(h) for h in hours.index[(hours["count"] >= MIN_VISITS_PER_HOUR) & (hours["mean"] >
overall_mean)]),
        "dept_median": train.groupby("service_department")[TARGET].median().to_dict(),
        "district_median": train.groupby("district")[TARGET].median().to_dict(),
        "overall_median": float(train[TARGET].median()),
        "month_mode": int(train["visit_date"].dt.month.mode()[0]),
        "level_median": train.groupby("facility_type")[TARGET].median().to_dict(),
        "level_dept_median": level_dept["median"][level_dept["count"] >= MIN_GROUP_SIZE].to_dict(),
    }


def add_features(data, p):
    """Build the new features. Uses only information known when the patient arrives."""
    f = pd.DataFrame(index=data.index)
    f["travel_speed_kmh"] = data["distance_to_facility_km"] / (data["estimated_travel_time_min"] / 60)
    f["age_group"] = pd.cut(data["patient_age_years"], AGE_BINS, labels=False).astype(int)
    f["visit_month"] = data["visit_date"].dt.month.fillna(p["month_mode"]).astype(int)
    f["is_weekend"] = data["day_of_week"].isin(["Saturday", "Sunday"]).astype(int)
    f["is_peak_hour"] = data["arrival_hour"].isin(p["peak_hours"]).astype(int)
    f["walkin_at_peak"] = ((data["appointment_type"] == "Walk-in") & (f["is_peak_hour"] == 1)).astype(int)
    f["is_first_visit"] = (data["previous_visits_12m"] == 0).astype(int)
    f["dept_median_wait"] = data["service_department"].map(p["dept_median"]).fillna(p["overall_median"])
    f["district_median_wait"] = data["district"].map(p["district_median"]).fillna(p["overall_median"])
    f["level_dept_median_wait"] = ((data["facility_type"] + " | " +
data["service_department"]).map(p["level_dept_median"])
                                   .fillna(data["facility_type"].map(p["level_median"])).fillna(p["overall_median"]))
    f["hospital_at_peak"] = ((data["facility_type"] != "Health Centre") & (f["is_peak_hour"] == 1)).astype(int)
    return f


feature_params = learn_feature_params(train_imp)
train_new, val_new, test_new = (add_features(s, feature_params) for s in (train_imp, val_imp, test_imp))
ENGINEERED = train_new.columns.tolist()

print("Peak hours learned from training:", feature_params["peak_hours"])
print("Most common month in training (fills a missing date):", feature_params["month_mode"])
print("Department medians learned from training:", feature_params["dept_median"])
print("Facility type x department groups with their own median:", len(feature_params["level_dept_median"]))
print("New features:", len(ENGINEERED))
display(train_new.head())
display(train_new.describe().T.round(2))
```

Output

```
Training mean waiting time: 40.6
```

arrival_hour	7.0	8.0	9.0	10.0	11.0	12.0	13.0	14.0	15.0	16.0
count	157	102	134	127	83	63	31	16	4	1
mean	43.0	47.0	45.7	37.4	35.8	31.0	33.4	31.9	47.5	49.0
peak hour?	True	True	True	False	False	False	False	False	False	False

```
Peak hours learned from training:
 [7, 8, 9]
Most common month in training (fills a missing date): 4
Department medians learned from training: {'Child Health': 39.0, 'Dental': 31.0, 'Emergency': 34.0, 'Eye Clinic': 33.0,
'General Outpatient': 45.0, 'Maternal Health': 43.5, 'NCD Clinic': 30.5}
```

```
Facility type x department groups with their own median: 15
New features: 11
             travel_speed_kmh  age_group  visit_month  is_weekend  is_peak_hour  walkin_at_peak  is_first_visit  dept_median_wait
district_median_wait  level_dept_median_wait  hospital_at_peak
visit_id
RW-HLTH-00959         11.785714          1            6           0             0               0               0              43.5
40.0                    39.5                 0
RW-HLTH-00381         16.000000          3           10           0             0               0               0              45.0
37.0                    38.0                 0
RW-HLTH-00963          7.680000          2           12           0             1               0               0              45.0
40.0                    54.0                 1
RW-HLTH-00036         16.000000          3            6           0             1               1               0              33.0
39.5                    26.5                 0
RW-HLTH-00713         11.720930          2           11           0             0               0               0              33.0
32.5                    26.5                 0
```

	count	mean	std	min	25%	50%	75%	max
travel_speed_kmh	718.0	14.25	7.97	2.18	9.53	12.68	16.72	68.4
age_group	718.0	2.05	1.05	0.00	1.00	2.00	3.00	4.0
visit_month	718.0	6.45	3.45	1.00	4.00	6.00	9.00	12.0
is_weekend	718.0	0.30	0.46	0.00	0.00	0.00	1.00	1.0
is_peak_hour	718.0	0.55	0.50	0.00	0.00	1.00	1.00	1.0
walkin_at_peak	718.0	0.27	0.44	0.00	0.00	0.00	1.00	1.0
is_first_visit	718.0	0.12	0.33	0.00	0.00	0.00	0.00	1.0
dept_median_wait	718.0	39.97	5.80	30.50	34.00	43.50	45.00	45.0
district_median_wait	718.0	39.40	4.51	28.50	37.00	39.50	41.50	48.0
level_dept_median_wait	718.0	39.49	10.54	21.00	33.00	38.00	43.50	72.0
hospital_at_peak	718.0	0.16	0.37	0.00	0.00	0.00	0.00	1.0

What this does. `learn_feature_params()` learns from the training set the hours with long waits, the median waiting time of each department and each district, and the most common month. `add_features()` then builds eleven new features from the imputed data (in real units, before scaling).

New feature	How it is created	Kind	Business question
`travel_speed_kmh`	distance / (travel time / 60)	ratio	BQ3 - how easy is it to reach the facility
`age_group`	age cut into 0-4, 5-17, 18-39, 40-59, 60+ (coded 0 to 4)	bins	BQ4 - the manager asks for results by age group
`visit_month`	month taken from `visit_date` (1 to 12); a missing date gets the most common training month	date part	BQ2 - season
`is_weekend`	1 if Saturday or Sunday	date part	BQ2 - day of the visit
`is_peak_hour`	1 if the arrival hour is one of the hours whose training mean wait is above the overall training mean (hours with at least 30 training visits)	bin learned from training	BQ2 - time of arrival
`walkin_at_peak`	1 if walk-in AND peak hour	interaction	BQ5 - entry route combined with time
`is_first_visit`	1 if `previous_visits_12m` is 0	count flag	BQ4 - new patients
`dept_median_wait`	median waiting time of the department in the training set	group average learned from training	BQ1 - department
`district_median_wait`	median waiting time of the district in the training set (a district not seen in training gets the overall training median)	group average learned from training	BQ3 - location
`level_dept_median_wait`	median waiting time of the same facility type and department in the training set (a pair with fewer than 5 training visits gets the median of its facility type)	group average learned from training	BQ1 - facility level and department together
`hospital_at_peak`	1 if the facility is a district or referral hospital AND peak hour	interaction	BQ1 and BQ2 - are peak hours worse at hospitals

What we found. The peak hours learned from training are 7, 8 and 9 (mean waits of 43.0, 47.0 and 45.7 minutes against an overall training mean of 40.6). The other hours either have a mean below the overall mean or fewer than 30 training visits (hours 15 and 16 have 5 visits together). 15 pairs of facility type and department have at least 5 training visits and get their own median.

What we decided.

- All eleven features use only information known at arrival. None uses `consultation_duration_min` or `attended`.
- The three group medians use the target, so they are learned from training records only and then looked up for validation and test. A caution: inside the training set each record's own waiting time is part of its group median, so their link with the target looks a little stronger in training than it will be on new data. We check them on the validation set in section 5.4.
- The thresholds (age bands, the 30-visit rule, the 5-visit rule, weekend = Saturday and Sunday) are our assumptions.

5.2 Skewness and transformations (training set)

Code - cell 38

```
train_X = pd.concat([train_enc, train_new], axis=1)
val_X = pd.concat([val_enc, val_new], axis=1)
test_X = pd.concat([test_enc, test_new], axis=1)

# Amount columns that can be transformed. arrival_hour and visit_month are clock/calendar values and age is not an
amount with a long tail by nature; they are measured but only transformed if the rule says so.
AMOUNT_COLS = ["distance_to_facility_km", "estimated_travel_time_min", "previous_visits_12m", "travel_speed_kmh",
"patient_age_years"]


def learn_transforms(train):
    """Rule fixed in advance: skewness above 1 -> try log(1+x); between 0.5 and 1 -> try square root.
    The transformation is kept only if it brings the skewness inside -0.5 to 0.5."""
    rule, tried = {}, {}
    for c in AMOUNT_COLS:
        skew = train[c].skew()
        how = "log" if skew > 1 else "sqrt" if skew > 0.5 else "none"
        after = np.log1p(train[c]).skew() if how == "log" else np.sqrt(train[c]).skew() if how == "sqrt" else skew
        tried[c] = (how, round(float(skew), 2), round(float(after), 2))
        rule[c] = how if abs(after) < 0.5 else "none"
    return rule, tried


def apply_transforms(data, rule):
    """Apply the transformations and rename the columns so that the name shows what was done."""
    data = data.copy()
    for c, how in rule.items():
        if how == "log":
            data[c + "_log"] = np.log1p(data.pop(c))
        elif how == "sqrt":
            data[c + "_sqrt"] = np.sqrt(data.pop(c))
    return data


transform_rule, transform_tried = learn_transforms(train_X)
train_T, val_T, test_T = (apply_transforms(s, transform_rule) for s in (train_X, val_X, test_X))

rows = []
for c, how in transform_rule.items():
    new = c + {"log": "_log", "sqrt": "_sqrt", "none": ""}[how]
    tried_how, before_skew, tried_skew = transform_tried[c]
    rows.append([c, before_skew, tried_how, tried_skew, how, new, round(train_T[new].skew(), 2)])
skew_table = pd.DataFrame(rows, columns=["Column", "Skewness before (train)", "Transformation tried", "Skewness if
transformed",
                                         "Transformation kept", "Final column", "Final skewness (train)"])
display(skew_table)

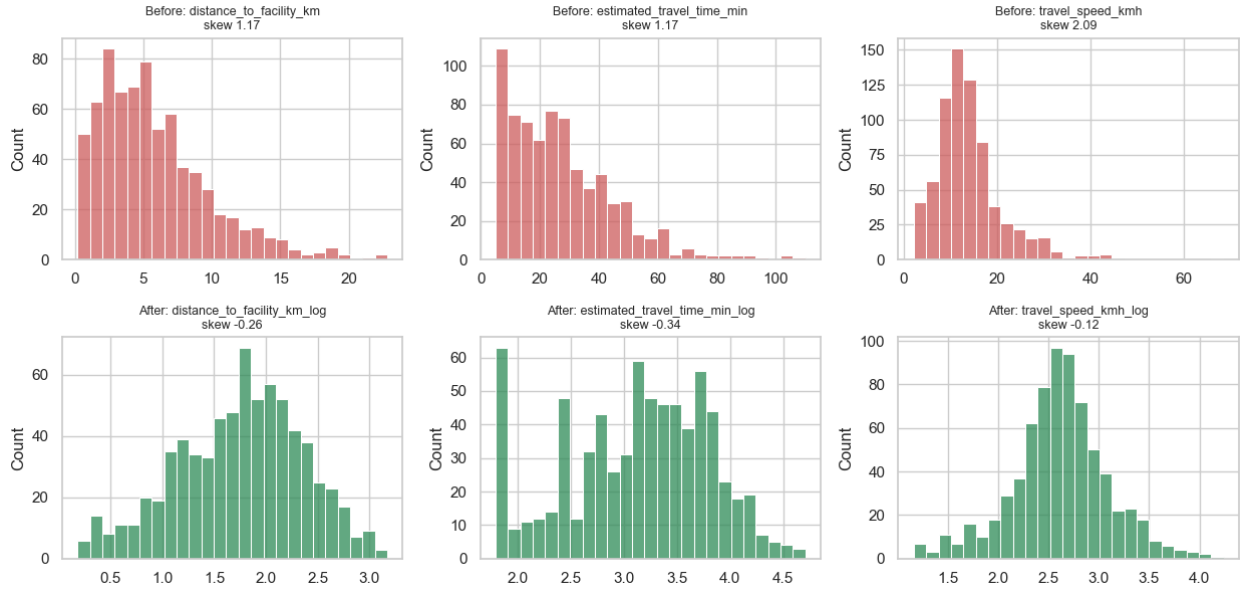
changed = [(c, c + ("_log" if h == "log" else "_sqrt")) for c, h in transform_rule.items() if h != "none"]
fig, axes = plt.subplots(2, len(changed), figsize=(4.2 * len(changed), 6.5), squeeze=False)
for j, (old, new) in enumerate(changed):
    sns.histplot(train_X[old], bins=25, ax=axes[0, j], color="indianred")
    axes[0, j].set_title(f"Before: {old}\nskew {train_X[old].skew():.2f}", fontsize=9)
    sns.histplot(train_T[new], bins=25, ax=axes[1, j], color="seagreen")
    axes[1, j].set_title(f"After: {new}\nskew {train_T[new].skew():.2f}", fontsize=9)
    axes[0, j].set_xlabel(""); axes[1, j].set_xlabel("")
fig.suptitle("Figure 8 - Skewed columns before (top) and after (bottom) transformation, training set")
plt.tight_layout()
plt.show()
```

Output

	Column	Skewness before	Transformation tried	Skewness if transformed	Transformation kept	Final column	Final skewness

		(train)					(train)
0	distance_to_facility_km	1.17	log	-0.26	log	distance_to_facility_km_log	-0.26
1	estimated_travel_time_min	1.17	log	-0.34	log	estimated_travel_time_min_log	-0.34
2	previous_visits_12m	0.76	sqrt	-0.62	none	previous_visits_12m	0.76
3	travel_speed_kmh	2.09	log	-0.12	log	travel_speed_kmh_log	-0.12
4	patient_age_years	0.29	none	0.29	none	patient_age_years	0.29

Figure 8 - Skewed columns before (top) and after (bottom) transformation, training set



What this does. Joins the encoded columns and the new features, measures the skewness of the amount columns on the training set and applies a rule fixed in advance: skewness above 1 tries `log(1 + x)`, skewness between 0.5 and 1 tries a square root, otherwise nothing. A transformation is kept only if it brings the skewness inside -0.5 to 0.5. Transformed columns are renamed with `_log` or `_sqrt`.

What we found (training set).

- Distance (skewness 1.17), travel time (1.17) and travel speed (2.09) are above 1. The log brings them to -0.26, -0.34 and -0.12, so the log is kept.
- `previous_visits_12m` (0.76) was tried with a square root, but the result (-0.62) is still outside -0.5 to 0.5: the tail simply moves to the left. The column is left as it is.
- Age (0.29) needs no transformation.

Figure 8 (before/after histograms) - interpretation. The three transformed columns lose their long right tail and become roughly symmetric: skewness goes from 1.17, 1.17 and 2.09 (top row) to -0.26, -0.34 and -0.12 (bottom row).

What we decided. The thresholds 0.5 and 1 are a common rule of thumb and are our assumption. `arrival_hour` and `visit_month` are clock and calendar values, so they are not transformed. The same transformations are applied to validation and test.

5.3 Scale the continuous columns (scaler fitted on the training set only)

Code - cell 39

```
BINARY_OR_ORDINAL = [c for c in train_T.columns if set(train_T[c].unique()) <= {0, 1}] + ["facility_level",
"age_group"]
SCALE_COLS = [c for c in train_T.columns if c not in BINARY_OR_ORDINAL]


def apply_scaler(data, scaler):
    data = data.copy()
    data[SCALE_COLS] = scaler.transform(data[SCALE_COLS])
    return data


scaler = StandardScaler().fit(train_T[SCALE_COLS])            # learned from training only
```

```
train_S, val_S, test_S = (apply_scaler(s, scaler) for s in (train_T, val_T, test_T))
y_train, y_val, y_test = train_imp[TARGET], val_imp[TARGET], test_imp[TARGET]      # the target is NOT scaled

print("Columns scaled:", SCALE_COLS)
check = pd.DataFrame({"train mean": train_S[SCALE_COLS].mean(), "train std": train_S[SCALE_COLS].std(ddof=0),
                      "validation mean": val_S[SCALE_COLS].mean(), "validation std":
val_S[SCALE_COLS].std(ddof=0)}).round(3)
display(check)
print("Target left in minutes: train mean", round(y_train.mean(), 2))
```

Output

```
Columns scaled: ['patient_age_years', 'arrival_hour', 'previous_visits_12m', 'visit_month', 'dept_median_wait',
'district_median_wait', 'level_dept_median_wait', 'distance_to_facility_km_log', 'estimated_travel_time_min_log',
'travel_speed_kmh_log']
```

	train mean	train std	validation mean	validation std
patient_age_years	0.0	1.0	0.056	0.998
arrival_hour	-0.0	1.0	-0.019	1.066
previous_visits_12m	-0.0	1.0	0.084	0.910
visit_month	-0.0	1.0	0.098	0.958
dept_median_wait	0.0	1.0	0.050	0.965
district_median_wait	-0.0	1.0	-0.126	1.038
level_dept_median_wait	-0.0	1.0	0.026	1.066
distance_to_facility_km_log	0.0	1.0	-0.009	0.931
estimated_travel_time_min_log	-0.0	1.0	-0.086	1.071
travel_speed_kmh_log	0.0	1.0	0.144	0.967

```
Target left in minutes: train mean 40.57
```

What this does. Standardises the continuous columns: subtract the training mean and divide by the training standard deviation. The scaler is fitted on the training set and then applied to all three sets. This is the scaling step required in Stage 4; it is placed here so that it comes after the new features and the transformations.

What we found. Ten columns are scaled. In the training set each has mean 0 and standard deviation 1. In the validation set the means lie between -0.13 and 0.14 and the standard deviations between 0.91 and 1.07: close to 0 and 1 but not exactly, which is expected and correct, because validation uses the training values and not its own.

What we decided. 0/1 columns and the two ordered codes (`facility_level` 0-2, age_group 0-4) are not scaled, so that they stay readable. The target stays in minutes.

5.4 Relationship with the target (training set): correlation and mutual information

Code - cell 40

```
discrete_mask = [c in BINARY_OR_ORDINAL for c in train_S.columns]
mi = mutual_info_regression(train_S, y_train, discrete_features=discrete_mask, random_state=SEED)
f_all, p_all = f_regression(train_S, y_train)

relation = pd.DataFrame({
    "Correlation with target (r)": train_S.corrwith(y_train),
    "Mutual information": mi,
    "p-value (F-test)": p_all,
    "Engineered": [c.replace("_log", "").replace("_sqrt", "") in ENGINEERED for c in train_S.columns],
}, index=train_S.columns)
relation["|r|"] = relation["Correlation with target (r)"].abs()
relation = relation.sort_values("|r|", ascending=False)
display(relation.drop(columns="|r|").round(4))

group_features = ["dept_median_wait", "district_median_wait", "level_dept_median_wait"]
val_check = pd.DataFrame({"r in training": train_S[group_features].corrwith(y_train),
                          "r in validation": val_S[group_features].corrwith(y_val)}).round(3)
print("Group-median features checked on the validation set (not the test set):")
display(val_check)
```

Output

	Correlation with target (r)	Mutual information	p-value (F-test)	Engineered
level_dept_median_wait	0.5087	0.1432	0.0000	True
facility_level	0.4373	0.0702	0.0000	False
hospital_at_peak	0.3869	0.0561	0.0000	True
is_peak_hour	0.2474	0.0144	0.0000	True
dept_median_wait	0.2358	0.0390	0.0000	True
arrival_hour	-0.2119	0.0462	0.0000	False
service_department_general_outpatient	0.1986	0.0200	0.0000	False
walkin_at_peak	0.1883	0.0000	0.0000	True
district_median_wait	0.1360	0.0166	0.0003	True
service_department_ncd_clinic	-0.1214	0.0000	0.0011	False

service_department_dental	-0.1198	0.0101	0.0013	False
service_department_emergency	-0.0921	0.0000	0.0135	False
day_of_week_monday	-0.0531	0.0000	0.1555	False
day_of_week_sunday	0.0521	0.0000	0.1629	False
appointment_type_walk_in	0.0514	0.0000	0.1693	False
insurance_type_none	0.0476	0.0124	0.2031	False
service_department_eye_clinic	-0.0473	0.0000	0.2054	False
province_kigali_city	0.0402	0.0141	0.2824	False
distance_to_facility_km_log	0.0395	0.0085	0.2900	False
is_weekend	0.0380	0.0003	0.3096	True
province_southern	0.0370	0.0000	0.3224	False
appointment_type_scheduled	-0.0360	0.0000	0.3357	False
appointment_type_referral	0.0341	0.0000	0.3612	False
day_of_week_thursday	-0.0312	0.0000	0.4040	False
day_of_week_wednesday	0.0291	0.0000	0.4356	False
estimated_travel_time_min_log	0.0252	0.0040	0.5008	False
previous_visits_12m	-0.0233	0.0055	0.5333	False
service_department_maternal_health	0.0213	0.0050	0.5691	False
sex_male	-0.0208	0.0216	0.5787	False
insurance_type_private	-0.0206	0.0067	0.5823	False
province_western	-0.0191	0.0000	0.6099	False
referral_yes	0.0161	0.0090	0.6660	False
is_first_visit	0.0150	0.0094	0.6883	True
day_of_week_tuesday	0.0136	0.0182	0.7151	False
travel_speed_kmh_log	0.0128	0.0000	0.7314	True
province_northern	-0.0121	0.0201	0.7456	False
age_group	0.0080	0.0000	0.8309	True
visit_month	-0.0071	0.0145	0.8490	True
day_of_week_saturday	-0.0058	0.0128	0.8775	False
patient_age_years	-0.0045	0.0000	0.9044	False
insurance_type_rssb	0.0025	0.0000	0.9477	False

`Group-median features checked on the validation set (not the test set):`

	r in training	r in validation
dept_median_wait	0.236	0.117
district_median_wait	0.136	0.120
level_dept_median_wait	0.509	0.427

What this does. Measures, on the training set, how each candidate feature relates to the target in two ways: the correlation (straight-line link, from -1 to 1) and mutual information (any kind of link, 0 = none). The p-value of the F-test says whether the straight-line link could be due to chance. No model is trained. The three group-median features are also checked on the validation set, because their training figure is slightly optimistic.

What we found (training set, 41 candidate features).

- The strongest link is the engineered `level_dept_median_wait` (r = 0.51, the highest mutual information, 0.14), followed by `facility_level` (r = 0.44), `hospital_at_peak` (r = 0.39), `is_peak_hour` (r = 0.25), `dept_median_wait` (r = 0.24) and `arrival_hour` (r = -0.21).
- 12 features have a p-value below 0.05. They all describe the facility level, the department or the time of arrival, plus the district median.
- Patient and access features show almost no link with waiting time in this data: age, age group, sex, insurance, previous visits, first visit, distance, travel time and travel speed all have |r| below 0.05 and p above 0.2. Month and weekend also show no link.
- All mutual information values are small (the highest is 0.14), which says that no single feature explains much of the waiting time on its own.
- Check on the validation set: `level_dept_median_wait` holds up (r = 0.51 in training, 0.43 in validation) and so does `district_median_wait` (0.14 and 0.12). `dept_median_wait` drops by half (0.24 to 0.12), so its training figure was optimistic.

What we decided. These figures are evidence only. The selection is made in the next section by a fixed rule. Mutual information has a small random part; it is fixed by the seed, so the table is the same on every run.

5.5 Feature selection (decided on the training set only)

Code - cell 41

```
ALPHA = 0.05            # rule for K: keep the features whose F-test p-value is below 0.05
CORR_LIMIT = 0.90

# Columns the business questions need, kept even if their score is low
ONEHOT_GROUPS = {col: [clean_name(f"{col}_{cat}") for cat in cats[1:]] for col, cats in zip(NOMINAL,
onehot_encoder.categories_)}
PROTECTED = {
    "service_department": "BQ1 - results are needed by department",
    "province": "BQ3 - results are needed by location",
    "appointment_type": "BQ5 - results are needed by entry route",
}
PROTECTED_SINGLE = {"age_group": "BQ4 - results are needed by age group", "arrival_hour": "BQ2 - results are needed by
time of arrival"}
protected_cols = [c for g in PROTECTED for c in ONEHOT_GROUPS[g]] + list(PROTECTED_SINGLE)

X = train_S.copy()
decision = {}

# Step 1 - constant columns
constant = [c for c in X.columns if X[c].nunique() <= 1]
for c in constant:
    decision[c] = "removed: constant"
X = X.drop(columns=constant)

# Step 2 - duplicated columns
duplicated = X.columns[X.T.duplicated()].tolist()
for c in duplicated:
    decision[c] = "removed: duplicate of another column"
X = X.drop(columns=duplicated)

# Step 3 - highly correlated pairs: drop the one with the weaker link to the target (never a protected column)
corr_matrix = X.corr().abs()
target_r = X.corrwith(y_train).abs()
pairs, dropped_corr = [], []
cols = X.columns.tolist()
for i, a in enumerate(cols):
    for b in cols[i + 1:]:
        if a in dropped_corr or b in dropped_corr or corr_matrix.loc[a, b] <= CORR_LIMIT:
            continue
        if (a in protected_cols) != (b in protected_cols):
            lose = b if a in protected_cols else a
        else:
            lose = a if target_r[a] < target_r[b] else b
        dropped_corr.append(lose)
        pairs.append([a, b, round(corr_matrix.loc[a, b], 3), lose])
        decision[lose] = f"removed: |r| > {CORR_LIMIT} with {b if lose == a else a}"
X = X.drop(columns=dropped_corr)

# Step 4 - SelectKBest with K fixed by the rule p < 0.05
f_scores, p_values = f_regression(X, y_train)
K = int((p_values < ALPHA).sum())
selector = SelectKBest(score_func=f_regression, k=K).fit(X, y_train)
selected = X.columns[selector.get_support()].tolist()

# Step 5 - keep whole one-hot groups and the columns the business questions need
final_features = []
for c in X.columns:
    group = next((g for g, members in ONEHOT_GROUPS.items() if c in members), None)
    if c in selected:
        decision[c] = f"kept: selected (p < {ALPHA})"
    elif c in protected_cols:
        why = PROTECTED.get(group) or PROTECTED_SINGLE.get(c)
        decision[c] = f"kept: business need ({why})"
    elif group and any(m in selected for m in ONEHOT_GROUPS[group]):
        decision[c] = "kept: completes a one-hot group that has a selected column"
    else:
        decision[c] = f"removed: not selected (p >= {ALPHA})"
        continue
    final_features.append(c)

print("Candidate features            :", train_S.shape[1])
print("Removed as constant           :", constant)
print("Removed as duplicated         :", duplicated)
print("Highly correlated pairs (|r| > 0.9):")
for a, b, r, lose in pairs:
    print(f"   {a}  ~  {b}  |r| = {float(r):.3f}  -> removed {lose}")
print(f"K chosen by the rule p < {ALPHA}  :", K)
print("Selected by SelectKBest       :", selected)
print("Final number of features      :", len(final_features))
```

```
after_corr = X.corr().abs()
print("Highest |r| between two features left after step 3:", round(float(after_corr.where(~np.eye(len(after_corr),
dtype=bool)).max().max()), 3))
print("Kept for business need or to complete a group:", [c for c in final_features if c not in selected])
engineered_kept = [c for c in final_features if c.replace("_log", "").replace("_sqrt", "") in ENGINEERED]
engineered_selected = [c for c in engineered_kept if c in selected]
print("Engineered features kept      :", engineered_kept)
print("  of which passed the statistical rule:", engineered_selected)
```

Output

```
Candidate features            : 41
Removed as constant           : []
Removed as duplicated         : []
Highly correlated pairs (|r| > 0.9):
   patient_age_years  ~  age_group  |r| = 0.956  -> removed patient_age_years
K chosen by the rule p < 0.05  : 12
Selected by SelectKBest       : ['arrival_hour', 'facility_level', 'service_department_dental',
'service_department_emergency', 'service_department_general_outpatient', 'service_department_ncd_clinic',
'is_peak_hour', 'walkin_at_peak', 'dept_median_wait', 'district_median_wait', 'level_dept_median_wait',
'hospital_at_peak']
Final number of features      : 22
Highest |r| between two features left after step 3: 0.835
Kept for business need or to complete a group: ['province_kigali_city', 'province_northern', 'province_southern',
'province_western', 'service_department_eye_clinic', 'service_department_maternal_health', 'appointment_type_referral',
'appointment_type_scheduled', 'appointment_type_walk_in', 'age_group']
Engineered features kept      : ['age_group', 'is_peak_hour', 'walkin_at_peak', 'dept_median_wait',
'district_median_wait', 'level_dept_median_wait', 'hospital_at_peak']
  of which passed the statistical rule: ['is_peak_hour', 'walkin_at_peak', 'dept_median_wait', 'district_median_wait',
'level_dept_median_wait', 'hospital_at_peak']
```

What this does. Selects the final features in five steps, using the training set only:

1. remove constant columns;
2. remove duplicated columns;
3. where two features are correlated above 0.90, remove the one less linked to the target;
4. `SelectKBest` with the F-test, where **K is not guessed**: K is the number of features with p < 0.05;
5. add back the columns the business questions need, and complete any one-hot group that has a selected column (a group with some of its columns missing would change the meaning of the reference category).

What we found.

- 41 candidate features. None is constant and none is duplicated.
- One pair is correlated above 0.90: `patient_age_years` and `age_group` (0.956). `patient_age_years` is removed, because `age_group` is the form the manager asked for (BQ4). After this step the highest correlation between two remaining features is 0.835.
- The rule p < 0.05 gives **K = 12**. SelectKBest keeps: `arrival_hour`, `facility_level`, four department columns (Dental, Emergency, General Outpatient, NCD Clinic), `is_peak_hour`, `walkin_at_peak`, `hospital_at_peak`, `dept_median_wait`, `district_median_wait` and `level_dept_median_wait`.
- 10 more columns are kept although their score is low: the four `province` columns (BQ3), the three `appointment_type` columns (BQ5), `age_group` (BQ4) and the two remaining department columns, Eye Clinic and Maternal Health, which complete the department group (BQ1).
- **Final: 22 features.** 19 are removed: `patient_age_years` (correlation) and 18 not selected (sex, the three insurance columns, the six day-of-week columns, referral, distance, travel time, previous visits, travel speed, month, weekend, first visit).

Why we keep low-score columns. The manager asked for results by department, location, entry route and age group. If those columns were dropped, the dataset could not answer BQ3, BQ4 and BQ5 at all. Their weak score is itself a finding: in this data, location, age and entry route show little link with waiting time. The analyst who trains the model can still drop them.

Honest statement on the engineered features. We created 11. Six pass the statistical rule (`is_peak_hour`, `walkin_at_peak`, `hospital_at_peak`, `dept_median_wait`, `district_median_wait`, `level_dept_median_wait`). `age_group` shows no link with the target (r = 0.008) and is kept only for the business question. Four are removed as not useful: `travel_speed_kmh`, `visit_month`, `is_weekend` and `is_first_visit`. Two of the six overlap with other columns: `walkin_at_peak` (r = 0.19) is weaker than `is_peak_hour` alone (r = 0.25), so most of its link comes from

the peak hour and not from the walk-in part; and `dept_median_wait` halves on the validation set. **How we got here (for transparency):** in our first pass we had nine engineered features and only four passed the rule. We then added two features that BQ1 and BQ2 call for (`level_dept_median_wait` and `hospital_at_peak`) and ran the same selection once more; both passed. We did not change the rule, the thresholds or the weak features in order to keep more of them.

5.6 Feature Justification table

Code - cell 42

```
HOW = {
    "patient_age_years": ("Original column, imputed (median), scaled", "BQ4"),
    "arrival_hour": ("Original column, scaled", "BQ2"),
    "distance_to_facility_km": ("Original column, imputed (median), transformed if skewed, scaled", "BQ3"),
    "estimated_travel_time_min": ("Original column, imputed (group median), transformed if skewed, scaled", "BQ3"),
    "previous_visits_12m": ("Original column, imputed (median), transformed if skewed, scaled", "BQ4"),
    "facility_level": ("Ordinal code of facility_type (0, 1, 2)", "BQ1"),
    "travel_speed_kmh": ("ENGINEERED ratio: distance / (travel time / 60)", "BQ3"),
    "age_group": ("ENGINEERED bins of age (0-4, 5-17, 18-39, 40-59, 60+)", "BQ4"),
    "visit_month": ("ENGINEERED date part: month of visit_date", "BQ2"),
    "is_weekend": ("ENGINEERED date part: Saturday or Sunday", "BQ2"),
    "is_peak_hour": ("ENGINEERED bin: arrival hour is a long-wait hour learned from training", "BQ2"),
    "walkin_at_peak": ("ENGINEERED interaction: walk-in AND peak hour", "BQ5"),
    "is_first_visit": ("ENGINEERED flag: previous_visits_12m = 0", "BQ4"),
    "dept_median_wait": ("ENGINEERED group average: training median wait of the department", "BQ1"),
    "district_median_wait": ("ENGINEERED group average: training median wait of the district", "BQ3"),
    "level_dept_median_wait": ("ENGINEERED group average: training median wait of the facility type x department",
"BQ1"),
    "hospital_at_peak": ("ENGINEERED interaction: hospital (not health centre) AND peak hour", "BQ1, BQ2"),
}
GROUP_BQ = {"province": "BQ3", "sex": "BQ4", "insurance_type": "BQ4", "service_department": "BQ1",
            "appointment_type": "BQ5", "day_of_week": "BQ2", "referral": "BQ5"}

rows = []
for c in train_S.columns:
    base = c.replace("_log", "").replace("_sqrt", "")
    group = next((g for g, members in ONEHOT_GROUPS.items() if c in members), None)
    how, bq = HOW[base] if base in HOW else (f"One-hot column of {group}", GROUP_BQ[group])
    evidence = (f"r = {relation.loc[c, 'Correlation with target (r)']:.3f}; MI = {relation.loc[c, 'Mutual
information']:.3f}; "
                f"p = {relation.loc[c, 'p-value (F-test)']:.4f}")
    if c in group_features:
        evidence += f"; validation r = {val_check.loc[c, 'r in validation']:.3f}"
    rows.append([c, how, bq, evidence, decision[c]])
feature_justification = pd.DataFrame(rows, columns=["Feature", "How created", "Business question", "Evidence (training
set)", "Decision"])
feature_justification.to_csv(DOCS_DIR / "feature_justification.csv", index=False)
pd.set_option("display.max_colwidth", 120)
display(feature_justification)

print("Engineered features:")
display(feature_justification[feature_justification["How created"].str.startswith("ENGINEERED")])
```

Output

	Feature	How created	Business question	Evidence (training set)	Decision
0	patient_age_years	Original column, imputed (median), scaled	BQ4	r = -0.004; MI = 0.000; p = 0.9044	removed: |r| > 0.9 with age_group
1	arrival_hour	Original column, scaled	BQ2	r = -0.212; MI = 0.046; p = 0.0000	kept: selected (p < 0.05)
2	previous_visits_12m	Original column, imputed (median), transformed if skewed, scaled	BQ4	r = -0.023; MI = 0.005; p = 0.5333	removed: not selected (p >= 0.05)
3	facility_level	Ordinal code of facility_type (0, 1, 2)	BQ1	r = 0.437; MI = 0.070; p = 0.0000	kept: selected (p < 0.05)
4	province_kigali_city	One-hot column of province	BQ3	r = 0.040; MI = 0.014; p = 0.2824	kept: business need (BQ3 - results are needed by location)
5	province_northern	One-hot column of province	BQ3	r = -0.012; MI = 0.020; p = 0.7456	kept: business need (BQ3 - results are needed by location)
6	province_southern	One-hot column of province	BQ3	r = 0.037; MI = 0.000; p = 0.3224	kept: business need (BQ3 - results are needed by location)
7	province_western	One-hot column of province	BQ3	r = -0.019; MI = 0.000; p = 0.6099	kept: business need (BQ3 - results are

					needed by location)
8	sex_male	One-hot column of sex	BQ4	r = -0.021; MI = 0.022; p = 0.5787	removed: not selected (p >= 0.05)
9	insurance_type_none	One-hot column of insurance_type	BQ4	r = 0.048; MI = 0.012; p = 0.2031	removed: not selected (p >= 0.05)
10	insurance_type_private	One-hot column of insurance_type	BQ4	r = -0.021; MI = 0.007; p = 0.5823	removed: not selected (p >= 0.05)
11	insurance_type_rssb	One-hot column of insurance_type	BQ4	r = 0.002; MI = 0.000; p = 0.9477	removed: not selected (p >= 0.05)
12	service_department_dental	One-hot column of service_department	BQ1	r = -0.120; MI = 0.010; p = 0.0013	kept: selected (p < 0.05)
13	service_department_emergency	One-hot column of service_department	BQ1	r = -0.092; MI = 0.000; p = 0.0135	kept: selected (p < 0.05)
14	service_department_eye_clinic	One-hot column of service_department	BQ1	r = -0.047; MI = 0.000; p = 0.2054	kept: business need (BQ1 - results are needed by department)
15	service_department_general_outpatient	One-hot column of service_department	BQ1	r = 0.199; MI = 0.020; p = 0.0000	kept: selected (p < 0.05)
16	service_department_maternal_health	One-hot column of service_department	BQ1	r = 0.021; MI = 0.005; p = 0.5691	kept: business need (BQ1 - results are needed by department)
17	service_department_ncd_clinic	One-hot column of service_department	BQ1	r = -0.121; MI = 0.000; p = 0.0011	kept: selected (p < 0.05)
18	appointment_type_referral	One-hot column of appointment_type	BQ5	r = 0.034; MI = 0.000; p = 0.3612	kept: business need (BQ5 - results are needed by entry route)
19	appointment_type_scheduled	One-hot column of appointment_type	BQ5	r = -0.036; MI = 0.000; p = 0.3357	kept: business need (BQ5 - results are needed by entry route)
20	appointment_type_walk_in	One-hot column of appointment_type	BQ5	r = 0.051; MI = 0.000; p = 0.1693	kept: business need (BQ5 - results are needed by entry route)
21	day_of_week_monday	One-hot column of day_of_week	BQ2	r = -0.053; MI = 0.000; p = 0.1555	removed: not selected (p >= 0.05)
22	day_of_week_saturday	One-hot column of day_of_week	BQ2	r = -0.006; MI = 0.013; p = 0.8775	removed: not selected (p >= 0.05)
23	day_of_week_sunday	One-hot column of day_of_week	BQ2	r = 0.052; MI = 0.000; p = 0.1629	removed: not selected (p >= 0.05)
24	day_of_week_thursday	One-hot column of day_of_week	BQ2	r = -0.031; MI = 0.000; p = 0.4040	removed: not selected (p >= 0.05)
25	day_of_week_tuesday	One-hot column of day_of_week	BQ2	r = 0.014; MI = 0.018; p = 0.7151	removed: not selected (p >= 0.05)
26	day_of_week_wednesday	One-hot column of day_of_week	BQ2	r = 0.029; MI = 0.000; p = 0.4356	removed: not selected (p >= 0.05)
27	referral_yes	One-hot column of referral	BQ5	r = 0.016; MI = 0.009; p = 0.6660	removed: not selected (p >= 0.05)
28	age_group	ENGINEERED bins of age (0-4, 5-17, 18-39, 40-59, 60+)	BQ4	r = 0.008; MI = 0.000; p = 0.8309	kept: business need (BQ4 - results are needed by age group)
29	visit_month	ENGINEERED date part: month of visit_date	BQ2	r = -0.007; MI = 0.015; p = 0.8490	removed: not selected (p >= 0.05)
30	is_weekend	ENGINEERED date part: Saturday or Sunday	BQ2	r = 0.038; MI = 0.000; p = 0.3096	removed: not selected (p >= 0.05)
31	is_peak_hour	ENGINEERED bin: arrival hour is a long-wait hour learned from training	BQ2	r = 0.247; MI = 0.014; p = 0.0000	kept: selected (p < 0.05)
32	walkin_at_peak	ENGINEERED interaction: walk-in AND peak hour	BQ5	r = 0.188; MI = 0.000; p = 0.0000	kept: selected (p < 0.05)
33	is_first_visit	ENGINEERED flag: previous_visits_12m = 0	BQ4	r = 0.015; MI = 0.009; p = 0.6883	removed: not selected (p >= 0.05)
34	dept_median_wait	ENGINEERED group average: training median wait of the department	BQ1	r = 0.236; MI = 0.039; p = 0.0000; validation r = 0.117	kept: selected (p < 0.05)
35	district_median_wait	ENGINEERED group average: training	BQ3	r = 0.136; MI = 0.017; p = 0.0003;	kept: selected (p < 0.05)

		median wait of the district		validation r = 0.120	
36	level_dept_median_wait	ENGINEERED group average: training median wait of the facility type x department	BQ1	r = 0.509; MI = 0.143; p = 0.0000; validation r = 0.427	kept: selected (p < 0.05)
37	hospital_at_peak	ENGINEERED interaction: hospital (not health centre) AND peak hour	BQ1, BQ2	r = 0.387; MI = 0.056; p = 0.0000	kept: selected (p < 0.05)
38	distance_to_facility_km_log	Original column, imputed (median), transformed if skewed, scaled	BQ3	r = 0.040; MI = 0.008; p = 0.2900	removed: not selected (p >= 0.05)
39	estimated_travel_time_min_log	Original column, imputed (group median), transformed if skewed, scaled	BQ3	r = 0.025; MI = 0.004; p = 0.5008	removed: not selected (p >= 0.05)
40	travel_speed_kmh_log	ENGINEERED ratio: distance / (travel time / 60)	BQ3	r = 0.013; MI = 0.000; p = 0.7314	removed: not selected (p >= 0.05)

`Engineered features:`

	Feature	How created	Business question	Evidence (training set)	Decision
28	age_group	ENGINEERED bins of age (0-4, 5-17, 18-39, 40-59, 60+)	BQ4	r = 0.008; MI = 0.000; p = 0.8309	kept: business need (BQ4 - results are needed by age group)
29	visit_month	ENGINEERED date part: month of visit_date	BQ2	r = -0.007; MI = 0.015; p = 0.8490	removed: not selected (p >= 0.05)
30	is_weekend	ENGINEERED date part: Saturday or Sunday	BQ2	r = 0.038; MI = 0.000; p = 0.3096	removed: not selected (p >= 0.05)
31	is_peak_hour	ENGINEERED bin: arrival hour is a long-wait hour learned from training	BQ2	r = 0.247; MI = 0.014; p = 0.0000	kept: selected (p < 0.05)
32	walkin_at_peak	ENGINEERED interaction: walk-in AND peak hour	BQ5	r = 0.188; MI = 0.000; p = 0.0000	kept: selected (p < 0.05)
33	is_first_visit	ENGINEERED flag: previous_visits_12m = 0	BQ4	r = 0.015; MI = 0.009; p = 0.6883	removed: not selected (p >= 0.05)
34	dept_median_wait	ENGINEERED group average: training median wait of the department	BQ1	r = 0.236; MI = 0.039; p = 0.0000; validation r = 0.117	kept: selected (p < 0.05)
35	district_median_wait	ENGINEERED group average: training median wait of the district	BQ3	r = 0.136; MI = 0.017; p = 0.0003; validation r = 0.120	kept: selected (p < 0.05)
36	level_dept_median_wait	ENGINEERED group average: training median wait of the facility type x department	BQ1	r = 0.509; MI = 0.143; p = 0.0000; validation r = 0.427	kept: selected (p < 0.05)
37	hospital_at_peak	ENGINEERED interaction: hospital (not health centre) AND peak hour	BQ1, BQ2	r = 0.387; MI = 0.056; p = 0.0000	kept: selected (p < 0.05)
40	travel_speed_kmh_log	ENGINEERED ratio: distance / (travel time / 60)	BQ3	r = 0.013; MI = 0.000; p = 0.7314	removed: not selected (p >= 0.05)

What this does. Builds the Feature Justification table for every candidate feature: how it was created, the business question it serves, the evidence from the training set and the decision. It is saved to `docs/feature_justification.csv`. The second table repeats the engineered features only.

What we found. Every one of the 41 candidate features has a line with its evidence and a decision: 12 "kept: selected", 10 kept for a business need or to complete a one-hot group, 1 removed for high correlation and 18

removed as not selected. Of the 11 engineered features, 7 are kept (6 selected, 1 for business need) and 4 are removed.

What we decided. The decisions are those of section 5.5; nothing is changed by hand in this table.

5.7 Apply identical steps to train, validation and test

Code - cell 43

```
train_final, val_final, test_final = (s[final_features].copy() for s in (train_S, val_S, test_S))

assert list(train_final.columns) == list(val_final.columns) == list(test_final.columns)
for name, s in (("train", train_final), ("validation", val_final), ("test", test_final)):
    print(f"{name}: {s.shape}, empty cells = {int(s.isna().sum().sum())}")
```

Output

```
train: (718, 22), empty cells = 0
validation: (154, 22), empty cells = 0
test: (155, 22), empty cells = 0
```

What this does. Keeps the same final columns, in the same order, in the three sets. Every earlier step (imputation, encoding, new features, transformation, scaling) was already applied to the three sets with the values learned from training.

What we found. The three sets have identical columns and no empty cells.

Stage 6 - Finalise, package and present (PC 1.1 - 1.4)

6.1 Save the final files and the fitted pipeline

Code - cell 44

```
TARGET_NAME = "target_waiting_time_min"
FINAL_FILES = {"train": PROCESSED_DIR / "train.csv", "validation": PROCESSED_DIR / "validation.csv", "test":
PROCESSED_DIR / "test.csv"}

for name, X_part, y_part in (("train", train_final, y_train), ("validation", val_final, y_val), ("test", test_final,
y_test)):
    out = X_part.copy()
    out[TARGET_NAME] = y_part
    out.to_csv(FINAL_FILES[name], index=False, encoding="utf-8")       # visit_id (identifier) is not written
    print(f"{name}: {out.shape} -> {FINAL_FILES[name].relative_to(ROOT)}")

pipeline = {
    "imputation": impute_params,
    "ordinal_encoder": ordinal_encoder,
    "onehot_encoder": onehot_encoder,
    "feature_params": feature_params,
    "transform_rule": transform_rule,
    "scale_cols": SCALE_COLS,
    "scaler": scaler,
    "final_features": final_features,
}
PIPELINE_FILE = PROCESSED_DIR / "preparation_pipeline.joblib"
joblib.dump(pipeline, PIPELINE_FILE)
print("Fitted pipeline ->", PIPELINE_FILE.relative_to(ROOT))
```

Output

```
train: (718, 23) -> data\processed\train.csv
validation: (154, 23) -> data\processed\validation.csv
test: (155, 23) -> data\processed\test.csv
Fitted pipeline -> data\processed\preparation_pipeline.joblib
```

What this does. Saves `train.csv`, `validation.csv` and `test.csv` to `data/processed/`. Each file holds the final features and the target, renamed `target_waiting_time_min` so that nobody can mistake it for a feature. `visit_id` is not written. Everything that was learned from the training set (fill values, encoders, peak hours, group medians, transformation rule, scaler, list of final features) is saved in one file with joblib: `preparation_pipeline.joblib`.

What we decided. The fitted pipeline is saved as a dictionary of learned values and fitted scikit-learn objects. It is used by the function `prepare_new_data()` below. Whoever loads the file needs that function from this notebook.

6.2 prepare_new_data(): from raw records to model-ready features

Code - cell 45

```
def prepare_new_data(raw_records, fitted):
    """Take raw visit records (same columns as the raw file) and return model-ready features.
    Uses only values stored in the fitted pipeline; nothing is learned from the new records."""
    data = tidy_text(raw_records)
    data = remove_duplicates(data)
    data = fix_types(data)
    data = repair_facility_type(data)
    data = impossible_to_missing(data)
    data = drop_leakage_and_identifiers(data)
    data = data.drop(columns=[TARGET], errors="ignore")                # new records do not have a waiting time yet
    data = apply_imputation(data, fitted["imputation"])
    features = pd.concat([apply_encoders(data, fitted["ordinal_encoder"], fitted["onehot_encoder"]),
                          add_features(data, fitted["feature_params"])], axis=1)
    features = apply_transforms(features, fitted["transform_rule"])
    features[fitted["scale_cols"]] = fitted["scaler"].transform(features[fitted["scale_cols"]])
    return features[fitted["final_features"]]


# The reusable function must reproduce the step-by-step result exactly, starting again from the RAW records
loaded = joblib.load(PIPELINE_FILE)
for name, expected in (("train", train_final), ("validation", val_final), ("test", test_final)):
    raw_part = raw[raw["visit_id"].isin(expected.index)]
    rebuilt = prepare_new_data(raw_part, loaded).loc[expected.index]
    pd.testing.assert_frame_equal(rebuilt, expected, check_exact=False, rtol=1e-12, atol=1e-12)
    print(f"{name}: prepare_new_data() reproduces the step-by-step result for {len(expected)} records")
```

Output

```
train: prepare_new_data() reproduces the step-by-step result for 718 records
```

```
validation: prepare_new_data() reproduces the step-by-step result for 154 records
test: prepare_new_data() reproduces the step-by-step result for 155 records
```

What this does. `prepare_new_data()` chains the same small functions used step by step above: tidy text, remove duplicates, fix types, rebuild facility type, impossible values to missing, drop leakage and identifiers, impute, encode, add features, transform, scale, keep the final columns. It learns nothing from the records it is given; every value comes from the saved pipeline.

What we found. Starting again from the raw records and the pipeline file loaded from disk, the function gives exactly the same values as the step-by-step work for all training, validation and test records. The `assert` would stop the notebook if a single value differed.

Code - cell 46

```
five_ids = test_final.index[:5]
five_raw = raw[raw["visit_id"].isin(five_ids)].drop(columns=[TARGET, "consultation_duration_min", "attended"])
print("Five raw records, as they would arrive (no waiting time, no consultation, no attendance):")
display(five_raw)
print("The same five records after prepare_new_data():")
display(prepare_new_data(five_raw, loaded))
```

Output

```
Five raw records, as they would arrive (no waiting time, no consultation, no attendance):
```

	visit_id	province	district	facility_code	facility_type	visit_date	patient_age_years	sex	insurance_type	service_department	appointment_type	arrival_hour	day_of_week
88	RW-HLTH-00432	Eastern	Bugesera	HC-BUG-01	Health Centre	2025-08-15	29.0	Male	RSSB	NCD Clinic	NaN	9	Friday
304	RW-HLTH-00277	Northern	Rulindo	HC-RUL-03	Health Centre	2024-01-09	28.0	Female	Mutuelle de Santé	Eye Clinic	Walk-in	11	Tuesday
490	RW-HLTH-00828	Kigali City	Kicukiro	HC-KIC-01	Health Centre	2024-12-17	24.0	Female	Mutuelle de Santé	General Outpatient	Scheduled	13	Tuesday
589	RW-HLTH-00302	Southern	Huye	HC-HUY-01	Health Centre	2024-12-24	48.0	Female	NaN	Maternal Health	Walk-in	8	Tuesday
835	RW-HLTH-00116	Northern	Burera	HC-BUR-03	Health Centre	2025-09-21	44.0	Male	RSSB	General Outpatient	Walk-in	11	Sunday

	distance_to_facility_km	estimated_travel_time_min	referral	previous_visits_12m
88	2.9	13.0	No	2.0
304	4.9	32.0	No	NaN
490	5.3	24.0	Yes	8.0
589	1.2	10.0	NO	2.0
835	3.1	25.0	No	3.0

```
The same five records after prepare_new_data():
```

visit_id	arrival_hour	facility_level	province_kigali_city	province_northern	province_southern	province_western	service_department_dental	service_department_emergency
RW-HLTH-00432	-0.215434	0	0	0	0	0	0	0
RW-HLTH-00277	0.812351	0	0	1	0	0	0	0
RW-HLTH-00828	1.840137	0	1	0	0	0	0	0
RW-HLTH-00302	-0.729327	0	0	0	1	0	0	0
RW-HLTH-00116	0.812351	0	0	1	0	0	0	0

visit_id	service_department_eye_clinic	service_department_general_outpatient	service_department_maternal_health	service_department_ncd_clinic	appointment_type_referral
RW-HLTH-00432	0	0	0	1	0
RW-HLTH-00277	1	0	0	0	0
RW-HLTH-00828	0	1	0	0	0
RW-HLTH-00302	0	0	1	0	0
RW-HLTH-00116	0	1	0	0	0

visit_id	appointment_type_scheduled	appointment_type_walk_in	age_group	is_peak_hour	walkin_at_peak	dept_median_wait	district_median_wait	level_dept_median_wait	hospital_at_peak
RW-HLTH-00432	0	1	2	1	1	-1.634447	-0.532684	-1.185807	0
RW-HLTH-00277	0	1	2	0	0	-1.202811	0.132862	-1.233271	0
RW-HLTH-00828	1	0	2	0	0	0.869044	1.242106	-0.141599	0
RW-HLTH-00302	0	1	3	1	1	0.610062	0.132862	0.000793	0
RW-HLTH-00116	0	1	3	0	0	0.869044	0.465635	-0.141599	0

What this does. Demonstrates the function on five raw records. The outcome columns are removed first, to show that the function works on records as they would look at arrival, before the wait is known.

What we found. The five raw rows, with their text labels and any gaps, come out as five rows of numbers with the final feature columns and no empty cells.

6.3 Definition of done (computed in code)

Code - cell 47

```
saved = {name: pd.read_csv(path) for name, path in FINAL_FILES.items()}
feature_names = [c for c in saved["train"].columns if c != TARGET_NAME]
clean_text = clean.select_dtypes("object")
banned = ["visit_id", "facility_code", "consultation_duration_min", "attended", TARGET]
n = sum(len(s) for s in saved.values())
shares = {name: len(s) / n for name, s in saved.items()}
ranges_ok = all(clean[c].dropna().between(low, high).all() for c, (low, high) in VALID_RANGES.items() if c in
clean.columns)
index_sets = [set(train_final.index), set(val_final.index), set(test_final.index)]

done = pd.DataFrame([
    ["No duplicates", "visit_id unique in the clean data; no record in two sets",
     clean.index.is_unique and not clean.reset_index().duplicated().any() and sum(len(s) for s in index_sets) ==
len(set.union(*index_sets))],
    ["Correct types", "every column of the final files is numerical",
     all(pd.api.types.is_numeric_dtype(s[c]) for s in saved.values() for c in s.columns)],
    ["Consistent labels", "no label differs only by spaces or upper/lower case in the clean data",
     all(clean_text[c].nunique() == clean_text[c].str.strip().str.lower().nunique() for c in clean_text.columns)],
    ["Realistic values", "all values inside the allowed ranges; target above 0",
     ranges_ok and all((s[TARGET_NAME] > 0).all() for s in saved.values())],
    ["No missing values", "0 empty cells in the final files", all(s.isna().sum().sum() == 0 for s in saved.values())],
    ["No leakage", "no leakage, identifier or raw target column among the features", not any(b in feature_names for b
in banned)],
    ["Split 70/15/15", f"shares {[round(100 * v, 1) for v in shares.values()]}; no record in two sets",
     abs(shares["train"] - 0.70) < 0.01 and abs(shares["validation"] - 0.15) < 0.01 and abs(shares["test"] - 0.15) <
0.01
     and not (index_sets[0] & index_sets[1]) and not (index_sets[0] & index_sets[2]) and not (index_sets[1] &
index_sets[2])],
    ["Same columns in all sets", f"{len(saved['train'].columns)} columns in the same order",
     list(saved["train"].columns) == list(saved["validation"].columns) == list(saved["test"].columns)],
    ["Five or more engineered features", f"{len(ENGINEERED)} created; {len(engineered_kept)} in the final files
({len(engineered_selected)} passed the statistical rule)",
     len(engineered_kept) >= 5],
    ["Scaler fitted on training only", "scaled training columns have mean 0",
     bool(np.allclose(train_final[[c for c in SCALE_COLS if c in final_features]].mean(), 0, atol=1e-9))],
    ["Target not scaled and clearly named", TARGET_NAME,
     TARGET_NAME in saved["train"].columns and bool(np.isclose(saved["train"][TARGET_NAME].mean(), y_train.mean()))],
    ["Raw file MD5 unchanged", md5(RAW_FILE), md5(RAW_FILE) == RAW_MD5_START == EXPECTED_RAW_MD5],
], columns=["Check", "How it is measured", "Passed"])
done["Result"] = done.pop("Passed").map({True: "PASS", False: "FAIL"})
done.to_csv(DOCS_DIR / "definition_of_done.csv", index=False)
display(done)
assert (done["Result"] == "PASS").all(), "A definition-of-done check failed."
print("All definition-of-done checks pass.")
```

Output

	Check	How it is measured	Result
0	No duplicates	visit_id unique in the clean data; no record in two sets	PASS
1	Correct types	every column of the final files is numerical	PASS
2	Consistent labels	no label differs only by spaces or upper/lower case in the clean data	PASS
3	Realistic values	all values inside the allowed ranges; target above 0	PASS
4	No missing values	0 empty cells in the final files	PASS
5	No leakage	no leakage, identifier or raw target column among the features	PASS
6	Split 70/15/15	shares [69.9, 15.0, 15.1]; no record in two sets	PASS
7	Same columns in all sets	23 columns in the same order	PASS
8	Five or more engineered features	11 created; 7 in the final files (6 passed the statistical rule)	PASS
9	Scaler fitted on training only	scaled training columns have mean 0	PASS
10	Target not scaled and clearly named	target_waiting_time_min	PASS
11	Raw file MD5 unchanged	fe54529b0294f43151a9f5b8815d01a7	PASS

```
All definition-of-done checks pass.
```

What this does. Reads the three final files back from disk and computes every definition-of-done check in code. The table is saved to `docs/definition_of_done.csv`. The `assert` stops the notebook if any check fails.

What we found. All twelve checks pass, including the raw file MD5, which is identical to the start of the notebook.

6.4 MD5 fingerprints of the final files

Code - cell 48

```
fingerprints = pd.DataFrame(
    [[p.relative_to(ROOT).as_posix(), p.stat().st_size, md5(p)] for p in [RAW_FILE, CLEAN_FILE,
*FINAL_FILES.values()]],
    columns=["File", "Bytes", "MD5"])
display(fingerprints)
for _, row in fingerprints.iterrows():
    print(row["MD5"], row["File"])
```

Output

	File	Bytes	MD5
0	data/raw/rwanda_health_facility_visits.csv	186300	fe54529b0294f43151a9f5b8815d01a7
1	data/interim/visits_clean.csv	144490	c98ad97751c16ea6d6daaa6ff695e660
2	data/processed/train.csv	86562	2b7fd861e6ce4f1426374014a4d2a8eb
3	data/processed/validation.csv	18957	553acb279e727f037d5d37dd9dcc39fd
4	data/processed/test.csv	19072	39a3cadc68fb8020cb548e2e10a4517e

```
fe54529b0294f43151a9f5b8815d01a7 data/raw/rwanda_health_facility_visits.csv
c98ad97751c16ea6d6daaa6ff695e660 data/interim/visits_clean.csv
2b7fd861e6ce4f1426374014a4d2a8eb data/processed/train.csv
553acb279e727f037d5d37dd9dcc39fd data/processed/validation.csv
39a3cadc68fb8020cb548e2e10a4517e data/processed/test.csv
```

What this does. Prints the MD5 fingerprint of the raw file, the clean file and the three final files.

How to use it. Run the whole notebook again (Kernel > Restart and Run All). The five fingerprints must be the same as before. The same fingerprints mean the same bytes, which proves that the preparation can be repeated. We ran the notebook twice before delivery and the fingerprints were identical.

6.5 Data Card

Code - cell 49

```
n_eng = len(engineered_kept)
data_card = f"""# Data Card - Patient waiting times, model-ready dataset (Q2)

**Group:** Group 4 - Jean Luc Tuyizere (26RP01265), Jean De Dieu Mugisha (26RP01151)

## Purpose
A model-ready dataset to predict `waiting_time_min` (regression): the minutes a patient waits before consultation at a
health facility. Prepared for an illustrative Ministry of Health scenario with a pilot in Nyabihu District. No model
has been trained.

## Source
File `rwanda_health_facility_visits.csv` (MD5 `{RAW_MD5_START}`). Source as stated by the group: NISR (National
Institute of Statistics of Rwanda). **This is not verified; link, date accessed and licence are TO BE COMPLETED BY THE
GROUP.** Until they are on file, the data must not be presented as verified Rwandan facility records. No second source
was considered.

## Size
- Raw: {raw.shape[0]:,} rows x {raw.shape[1]} columns ({parsed.min().date()} to {parsed.max().date()}, {n_districts}
districts, 5 provinces).
- Clean (`data/interim/visits_clean.csv`): {clean.shape[0]:,} rows x {clean.shape[1]} columns.
- Final (`data/processed/`): train {len(train_final)}, validation {len(val_final)}, test {len(test_final)} rows;
{len(final_features)} features + 1 target.

## Columns
- **Target:** `{TARGET_NAME}` (minutes, not scaled, not transformed).
- **Features ({len(final_features)}):** {", ".join(f"`{c}`" for c in final_features)}.
- Continuous columns are standardised with the training mean and standard deviation. 0/1 columns and the codes
`facility_level` (0-2) and `age_group` (0-4) are not scaled.

## Cleaning steps
1. Tidied text (spaces, upper/lower case). 2. Removed {full_dups} duplicate rows. 3. Converted `visit_date` to a date.
4. Rebuilt 10 empty `facility_type` values from the facility code prefix. 5. Checked for impossible values (none
found). 6. Kept all outliers (all are possible values). 7. Dropped leakage columns (`consultation_duration_min`,
`attended`) and identifiers (`visit_id`, `facility_code`). 8. Removed {int(raw.drop_duplicates()[TARGET].isna().sum())}
records with no target. 9. Split 70/15/15 (seed {SEED}, stratified by `facility_type`). 10. Imputed with training
medians, modes and a group median. 11. Encoded (ordinal for facility level, one-hot for the rest).
```

```
## Feature steps
{len(ENGINEERED)} features were engineered; {n_eng} are in the final files ({", ".join(engineered_kept)}), of which
{len(engineered_selected)} passed the statistical rule and the others are kept for a business question. Skewed amount
columns were transformed (rule: skewness > 1 log, 0.5-1 square root, kept only if the result is inside -0.5 to 0.5).
Selection on the training set: constant, duplicated and |r| > {CORR_LIMIT} filters, then SelectKBest (F-test) with K =
number of features with p < {ALPHA} (K = {K}); columns needed by the business questions were kept.

## Limitations and bias risks
- **Source not verified.** The stated source is NISR, with no link or licence on file. There are signs that the file
may be generated (record codes run exactly 1 to 1,200; fixed facility-code pattern). Applying results to Rwanda is a
risk until the source is confirmed.
- **Small sample.** {clean.shape[0]:,} usable records, about {round(clean.shape[0] / n_districts)} per district;
referral hospitals have only {int((clean['facility_type'] == 'Referral Hospital').sum())} records.
- **Illogical records kept.** {int(logic_table.loc[7, 'Records'])} Child Health visits with a patient aged 18+, and
{int(logic_table.loc[8, 'Records'])} Maternal Health visits with a male patient (counts in the raw file).
- **Survivorship.** Visits where the patient did not attend have no waiting time and are excluded, so the data
describes only patients who stayed.
- **Assumptions.** All column meanings and allowed ranges are assumed. `None` in `insurance_type` is read as "no
insurance".
- **Coverage.** {n_districts} of Rwanda's 30 districts appear. The three group-median features use the target (training
records only) and look slightly stronger in training than on new data.
- **Privacy.** No names, phone numbers or national IDs. Identifier columns are removed from the final files.

## How to reproduce
Python 3.13 with pandas, numpy, matplotlib, seaborn, scikit-learn and joblib. Open
`notebooks/q2_waiting_time_data_preparation.ipynb` and choose Restart and Run All. The notebook reads
`data/raw/{RAW_NAME}` (never edited), writes the clean file, the three final files and `preparation_pipeline.joblib`,
and prints MD5 fingerprints that must match on every run. Use `prepare_new_data(raw_records, joblib.load(...))` for new
records.
"""
(DOCS_DIR / "data_card.md").write_text(data_card, encoding="utf-8")
display(Markdown(data_card))
```

Output (rendered document)

Data Card - Patient waiting times, model-ready dataset (Q2)

Group: Group 4 - Jean Luc Tuyizere (26RP01265), Jean De Dieu Mugisha (26RP01151)

Purpose

A model-ready dataset to predict `waiting_time_min` (regression): the minutes a patient waits before consultation at a health facility. Prepared for an illustrative Ministry of Health scenario with a pilot in Nyabihu District. No model has been trained.

Source

File `rwanda_health_facility_visits.csv` (MD5 `fe54529b0294f43151a9f5b8815d01a7`). Source as stated by the group: NISR (National Institute of Statistics of Rwanda). **This is not verified; link, date accessed and licence are TO BE COMPLETED BY THE GROUP.** Until they are on file, the data must not be presented as verified Rwandan facility records. No second source was considered.

Size

- Raw: 1,200 rows x 20 columns (2024-01-02 to 2025-12-30, 22 districts, 5 provinces).
- Clean (`data/interim/visits_clean.csv`): 1,027 rows x 16 columns.
- Final (`data/processed/`): train 718, validation 154, test 155 rows; 22 features + 1 target.

Columns

- **Target:** `target_waiting_time_min` (minutes, not scaled, not transformed).
- **Features (22):** `arrival_hour`, `facility_level`, `province_kigali_city`, `province_northern`, `province_southern`, `province_western`, `service_department_dental`, `service_department_emergency`, `service_department_eye_clinic`, `service_department_general_outpatient`, `service_department_maternal_health`, `service_department_ncd_clinic`, `appointment_type_referral`, `appointment_type_scheduled`, `appointment_type_walk_in`, `age_group`, `is_peak_hour`, `walkin_at_peak`, `dept_median_wait`, `district_median_wait`, `level_dept_median_wait`, `hospital_at_peak`.

- Continuous columns are standardised with the training mean and standard deviation. 0/1 columns and the codes `facility_level` (0-2) and `age_group` (0-4) are not scaled.

Cleaning steps

1. Tidied text (spaces, upper/lower case). 2. Removed 5 duplicate rows. 3. Converted `visit_date` to a date. 4. Rebuilt 10 empty `facility_type` values from the facility code prefix. 5. Checked for impossible values (none found). 6. Kept all outliers (all are possible values). 7. Dropped leakage columns (`consultation_duration_min`, `attended`) and identifiers (`visit_id`, `facility_code`). 8. Removed 168 records with no target. 9. Split 70/15/15 (seed 42, stratified by `facility_type`). 10. Imputed with training medians, modes and a group median. 11. Encoded (ordinal for facility level, one-hot for the rest).

Feature steps

11 features were engineered; 7 are in the final files (age_group, is_peak_hour, walkin_at_peak, dept_median_wait, district_median_wait, level_dept_median_wait, hospital_at_peak), of which 6 passed the statistical rule and the others are kept for a business question. Skewed amount columns were transformed (rule: skewness > 1 log, 0.5-1 square root, kept only if the result is inside -0.5 to 0.5). Selection on the training set: constant, duplicated and $|r| > 0.9$ filters, then SelectKBest (F-test) with K = number of features with $p < 0.05$ (K = 12); columns needed by the business questions were kept.

Limitations and bias risks

- **Source not verified.** The stated source is NISR, with no link or licence on file. There are signs that the file may be generated (record codes run exactly 1 to 1,200; fixed facility-code pattern). Applying results to Rwanda is a risk until the source is confirmed.
- **Small sample.** 1,027 usable records, about 47 per district; referral hospitals have only 30 records.
- **Illogical records kept.** 105 Child Health visits with a patient aged 18+, and 77 Maternal Health visits with a male patient (counts in the raw file).
- **Survivorship.** Visits where the patient did not attend have no waiting time and are excluded, so the data describes only patients who stayed.
- **Assumptions.** All column meanings and allowed ranges are assumed. `None` in `insurance_type` is read as "no insurance".
- **Coverage.** 22 of Rwanda's 30 districts appear. The three group-median features use the target (training records only) and look slightly stronger in training than on new data.
- **Privacy.** No names, phone numbers or national IDs. Identifier columns are removed from the final files.

How to reproduce

Python 3.13 with pandas, numpy, matplotlib, seaborn, scikit-learn and joblib. Open `notebooks/q2_waiting_time_data_preparation.ipynb` and choose Restart and Run All. The notebook reads `data/raw/rwanda_health_facility_visits.csv` (never edited), writes the clean file, the three final files and `preparation_pipeline.joblib`, and prints MD5 fingerprints that must match on every run. Use `prepare_new_data(raw_records, joblib.load(...))` for new records.

What this does. Writes the one-page Data Card, with every number taken from the variables of this notebook so that the card always matches the real result. It is saved to `docs/data_card.md`.

Summary of deliverables

Deliverable	Where
Environment (library versions)	Stage 1 of this notebook; `requirements.txt`
Scenario brief, proposal form, business questions	Stage 0 of this notebook; `docs/scenario_brief.md`, `docs/scenario_proposal_form.md`
Requirements, source comparison, requirement check, Source Log	Stage 2; `docs/source_log.md`
Data Dictionary	`docs/data_dictionary.csv`
Data Quality Report	`docs/data_quality_report.csv`
Cleaning Log	`docs/cleaning_log.csv`
Feature Justification table	`docs/feature_justification.csv`
Definition of done	`docs/definition_of_done.csv`
Data Card	`docs/data_card.md`
Raw, clean and final data	`data/raw/`, `data/interim/visits_clean.csv`, `data/processed/train.csv`, `validation.csv`, `test.csv`
Fitted pipeline	`data/processed/preparation_pipeline.joblib`

Still to be completed by the group before submission: the link, date accessed and licence of the NISR source; a real second source for the comparison table; the lecturer's signature on the Scenario Proposal Form.

Final deliverables checklist

The table lists every file of the package as found on disk when this report was built.

Deliverable	File (inside q2_own_scenario/)	Status
Notebook (all stages, delivered without outputs)	notebooks/q2_waiting_time_data_preparation.ipynb	Present (154,591 bytes)
Environment file	requirements.txt	Present (183 bytes)
Raw data (unchanged)	data/raw/rwanda_health_facility_visits.csv	Present (186,300 bytes)
Clean data	data/interim/visits_clean.csv	Present (144,490 bytes)
Training set	data/processed/train.csv	Present (86,562 bytes)
Validation set	data/processed/validation.csv	Present (18,957 bytes)
Test set	data/processed/test.csv	Present (19,072 bytes)
Fitted pipeline (joblib)	data/processed/preparation_pipeline.joblib	Present (7,596 bytes)
Scenario brief and business questions	docs/scenario_brief.md	Present (3,442 bytes)
Scenario Proposal Form	docs/scenario_proposal_form.md	Present (2,205 bytes)
Source Log and source comparison	docs/source_log.md	Present (3,218 bytes)
Data Dictionary	docs/data_dictionary.csv	Present (2,370 bytes)
Data Quality Report	docs/data_quality_report.csv	Present (2,454 bytes)
Cleaning Log	docs/cleaning_log.csv	Present (2,518 bytes)
Feature Justification table	docs/feature_justification.csv	Present (5,871 bytes)
Definition of done	docs/definition_of_done.csv	Present (921 bytes)
Data Card	docs/data_card.md	Present (4,795 bytes)
Documentation report (this document)	docs/Q2_Documentation_Report.docx	Present

Open items (left blank on purpose).

Item	Where	Status
Link, date accessed and licence of the data source	Source Log, Scenario Proposal Form, Data Card	To be completed by the group
Second data source for the comparison table	Section 2.2	None considered; the task asks for two
Lecturer approval signature	Scenario Proposal Form	To be signed by the lecturer